

第一部分：各章节核心知识点梳理

【第1章】 导论 (Introduction)

1. 什么是软件? (What is Software?)

- **定义**: 不仅仅是计算机程序 (Computer programs), 还包括相关的文档 (Associated documentation)、库 (Libraries)、支持网站 (Support websites) 和配置数据 (Configuration data)。
- **区别**:
 - **通用产品 (Generic products)**: 为公开市场开发, 由开发者定义需求 (如 Office, CAD)。
 - **定制产品 (Customized products)**: 为特定客户开发, 由客户定义需求 (如 嵌入式控制系统)。

2. 什么是软件工程? (What is Software Engineering?)

- **定义**: 软件工程是一门工程学科 (Engineering discipline), 它关注软件生产的所有方面 (All aspects of software production), 从最初的构思到系统投入使用后的维护。
- **核心误区**: 软件工程不仅仅是写代码 (Programming), 它还包括项目管理、工具开发、方法论等。
- **职业软件开发 (Professional Software Development)**:
 - 不仅仅是写个程序 (Individual programming)。
 - 它是为他人使用而开发的, 通常由团队开发, 并且在其生命周期中不断维护和演化。

3. 好软件的 4 个本质属性 (Essential Attributes)

所有专业软件都必须具备以下四个属性 (必考点):

1. **可维护性 (Maintainability)**: 软件应能随着需求变化而进化。这是最重要的属性, 因为环境在变。
2. **可依赖性与安全性 (Dependability and Security)**: 软件不应造成物理或经济损失, 不应被恶意破坏 (包括 Reliability, Security, Safety)。
3. **效率 (Efficiency)**: 不浪费系统资源 (内存、处理器周期等), 包括响应时间、处理时间等。
4. **可接受性 (Acceptability)**: 必须能被用户理解、易用, 并兼容其它系统。

4. 软件工程的 4 个基本活动 (Fundamental Activities)

es)

无论采用哪种过程模型, 这四个活动都是核心:

1. **软件规格说明 (Software Specification)**: 定义软件应该做什么以及开发的约束。
2. **软件开发 (Software Development)**: 设计和实现 (Design and implementation) 软件。
3. **软件确认 (Software Validation)**: 检查软件是否满足客户需求 (测试)。
4. **软件演化 (Software Evolution)**: 修改软件以适应变化的客户和市场需求。

5. 影响软件的一般问题 (General Issues)

- **异构性 (Heterogeneity)**: 系统需在不同类型的网络和设备上运行。
- **商业和社会变革 (Business and Social Change)**: 需求快速变化, 需要快速开发。
- **安全和信任 (Security and Trust)**: 软件渗透到生活中, 必须可信。
- **规模 (Scale)**: 从极小的嵌入式系统到基于云的全球系统。

6. 软件工程职业道德 (Ethics)

软件工程师必须遵守的职业责任:

- **保密性 (Confidentiality)**: 尊重雇主或客户的机密。
- **能力 (Competence)**: 不接受超出自己能力范围的工作, 不虚报能力。
- **知识产权 (Intellectual Property Rights)**: 了解并保护专利、版权等。
- **计算机滥用 (Computer Misuse)**: 不利用技术技能滥用他人计算机 (如传播病毒、玩游戏等)。

【第2章】 软件过程 (Software Processes)

1. 软件过程模型 (Process Models)

- **瀑布模型 (Waterfall Model)**:
 - **特点**: 计划驱动 (Plan-driven), 阶段分明 (需求->设计->实现->测试->维护)。
 - **缺点**: 不灵活, 难以应对需求变更 (Difficult to accommodate change)。
 - **适用**: 需求非常明确的系统、大型工程项目、关键系统 (Critical systems)。
- **增量式开发 (Incremental Development)**:

- **特点**: 规格说明、开发和确认是交织在一起的 (Interleaved)。
- **优点**: 降低需求变更成本, 更易获得客户反馈, 能够更早交付 (Rapid delivery)。
- **缺点**: 过程不可见 (Process is not visible), 系统结构随增量增加而退化 (Structure degrades)。
- **集成与配置 (Integration and Configuration)**:
 - 基于软件复用 (Software Reuse)。使用 COTS (Commercial Off-The-Shelf) 系统进行配置和集成。

2. 应对变更的两种方法 (Coping with Change)

- **变更预见 (Change Anticipation)**:
 - **方法**: **原型 (System Prototyping)**。在开发前快速构建版本以演示概念, 帮助明确需求。
- **变更容忍 (Change Tolerance)**:
 - **方法**: **增量式交付 (Incremental Delivery)**。分批次交付功能, 早期增量包含高优先级需求。

3. 测试阶段 (Stages of Testing)

1. **组件测试 (Component Testing)**: 测试独立的程序组件。
2. **系统测试 (System Testing)**: 测试整个集成后的系统。
3. **客户测试 (Customer Testing)**:
 - **Beta 测试 (Beta Testing)**: 由选定的用户 (Selected users) 使用真实数据进行测试。

4. 过程改进 (Process Improvement)

- **敏捷方法 (Agile Approach)**: 关注减少开销, 快速交付。
- **过程成熟度方法 (Process Maturity Approach/CMMI)**: 关注改进过程管理。
 - **SEI CMMI 5 个等级**:
 1. Initial (初始级)
 2. Managed (被管理级)
 3. Defined (被定义级)
 4. Quantitatively Managed (定量管理级)
 5. Optimizing (优化级)
- **改进循环**: 测量 (Measure) -> 分析 (Analyze) -> 变更 (Change)。

【第3章】 敏捷软件开发 (Agile Software Development)

1. 敏捷宣言 (Agile Manifesto)

- **个体和互动 (Individuals and interactions)** 胜过 流程和工具。
- **可以工作的软件 (Working software)** 胜过 详尽的文档。
- **客户协作 (Customer collaboration)** 胜过 合同谈判。
- **响应变化 (Responding to change)** 胜过 遵循计划。

2. 极限编程 (Extreme Programming, XP)

一种极具影响力的敏捷方法，核心实践包括：

- **用户故事 (User Stories)**：用场景代替文档化的需求规格说明。
- **结对编程 (Pair Programming)**：两人共用一台电脑，互相检查代码，支持代码共同拥有权 (Collective Ownership)。
- **重构 (Refactoring)**：持续改进代码结构，保持代码简单。
- **测试先行开发 (Test-first Development)**：在写代码前先写自动化测试用例。
- **持续集成 (Continuous Integration)**：任务一完成就集成，并运行所有测试。

3. Scrum 方法

一种敏捷项目管理框架。

- **角色**：
 - **Product Owner (产品负责人)**：确定需求优先级，管理 Product Backlog。
 - **ScrumMaster**：确保团队遵循 Scrum 流程，隔离外部干扰。
 - **Development Team**：自组织团队。
- **核心术语**：
 - **Sprint (冲刺)**：一个固定的开发周期（如 2-4 周）。
 - **Product Backlog**：待办事项清单。
 - **Daily Scrum**：每日站立会议。

第二部分：习题解析 (Quiz Review)

以下题目解析基于上述知识点整理。

1. 判断题

【第 1 章】 There are universal notations, methods, or techniques for software engineering because different types of software require the same approaches.
(软件工程有通用的符号、方法或技术，因为不同类型的软件需要相同的方法。)

要相同的方法。)

- **答案**： False (✗)
- **知识点**： **Software Engineering Diversity (软件工程多样性)**
 - 软件工程没有通用的符号、方法或技术 (No universal notations, methods, or techniques)。
 - 因为不同类型的软件（如游戏 vs 安全关键系统）需要不同的方法 (Different types of software require different approaches)。
- **解析**：题目表述正好与事实相反。不同类型的应用决定了应该使用哪种软件工程技术，没有一种方法是万能的。

2. 填空题

【第 1 章】 Many software failures are a consequence of two factors, one is increasing system complexity, and the other is failure to use [填空 1] methods.
(许多软件故障是两个因素造成的，一个是系统复杂性增加，另一个是未能使用___方法。)

- **答案**： software engineering / 软件工程 / SE
- **知识点**： **Software Project Failure (软件项目失败)**
 - 失败原因：
 1. Increasing system complexity (系统复杂性增加)。
 2. Failure to use **software engineering** methods (未能使用软件工程方法)。
- **解析**：很多公司在开发时没有遵循系统的工程化方法，导致软件昂贵且不可靠。

3. 判断题

【第 1 章】 Software engineering is intended to support both professional software development and individual programming.
(软件工程旨在支持专业软件开发和个人编程。)

- **答案**： False (✗)
- **知识点**： **Professional Software vs Individual Programming (职业软件 vs 个人编程)**
 - 软件工程旨在支持 Professional software development (专业软件开发)，而不是 Individual programming (个人编程)。
- **解析**：个人编程通常不需要复杂的文档、配置管理或演化策略，而这些都是软件工程的核心内容。

4. 填空题

【第 1 章】 The fundamental software engineering activities include software specification, software design and development, s

oftware validation and [填空 1].

(基本的软件工程活动包括软件规格说明、软件设计与开发、软件确认和___。)

- **答案**： software evolution / 软件演化 / 软件进化 / software maintenance / 软件维护
- **知识点**： **Software Process Activities (软件过程活动)**
 - 四个基本活动：Specification (规格说明), Development (开发), Validation (确认), **Evolution (演化)**。
- **解析**：Evolution 是指软件为了适应变化的客户需求和市场需求而进行的修改，是生命周期的重要一环。

5. 多选题

【第 1 章】 What are the essential attributes of good software?
(好软件的本质属性是什么？)

- A. Acceptability
- B. Dependability and security
- C. Efficiency
- D. Maintainability
- **答案**： A, B, C, D (全选)
- **知识点**： **Essential Attributes of Good Software (好软件的本质属性)**
 - Maintainability (可维护性)
 - Dependability and security (可依赖性与安全性)
 - Efficiency (效率)
 - Acceptability (可接受性)
- **解析**：这是第 1 章必须背诵的四个属性，缺一不可。

6. 多选题

【第 1 章】 What are included in a professional software?
(专业软件包含什么？)

- A. programs
- B. associated documentation
- C. libraries
- D. configuration data
- **答案**： A, B, C, D (全选)
- **知识点**： **What is Software? (什么是软件？)**
 - 软件不仅仅是程序 (Programs)。
 - 还包括： **Associated documentation** (文档), **Libraries** (库), **Configuration data** (配置数据), Support websites 等。
- **解析**：专业软件是一个完整的包，不仅要有代码，还要有

让代码能跑起来并被人理解的附属物。

7. 填空题

【第1章】 For generic products, the specification of what the software should do is owned by the software developer, while for customized products, the specification of what the software should do is owned by the [填空 1].

(对于通用产品，软件该做什么的规格说明归软件开发者所有；而对于定制产品，归____所有。)

- **答案：** customer / 客户
- **知识点：** Software Products (软件产品类型)
 - Generic products: Specification owned by developer.
 - Customized products: Specification owned by customer.
- **解析：** 定制软件是客户出钱委托开发的，因此“做什么”由客户说了算。

8. 判断题

Software engineering is an engineering discipline that is concerned solely with design and implementation of software product ion.

(软件工程是一门工程学科，仅关注软件生产的设计和实现。)

- **答案：** False (X)
- **知识点：** Software Engineering Definition (软件工程定义)
 - 软件工程关注软件生产的 all aspects (所有方面)。
 - 不仅是技术流程 (design and implementation)，还包括项目管理 (project management)、工具开发、方法论等。
- **解析：** 关键词是 "solely" (仅仅)。软件工程涵盖了从构思到维护的全过程，不只是写代码。

9. 填空题

【第1章】 According to the textbook, there are four related issues that affect many different types of software: heterogeneity, business and social change, security and trust, and [填空 1]. (教材指出，影响许多不同类型软件的四个相关问题是：异构性、商业和社会变革、安全和信任，以及____。)

- **答案：** scale / 规模
- **知识点：** General Issues Affecting Software (影响软件的一般问题)
 - Heterogeneity (异构性)

- Business and social change (商业和社会变革)
- Security and trust (安全和信任)
- Scale (规模)

- **解析：** Scale 指的是软件需要在极小的嵌入式设备到极大的云端集群等各种规模下运行。

10. 单选题

【第1章】 What is the most significant factor in determining which software engineering methods and techniques are most important?

(决定哪些软件工程方法和技术最重要的最关键因素是什么？)

- A. The customer
- B. The type of application being developed
- C. The manager
- D. The development team
- **答案：** B
- **知识点：** Software Engineering Diversity (软件工程多样性)
 - 没有通用的方法。
 - 决定性因素是 The type of application being developed (正在开发的应用程序类型)。
- **解析：** 开发一个简单的手机游戏和一个控制飞机的安全关键系统，所需要的方法和严谨程度是完全不同的。

11. 单选题

【第1章】 Which statement is false about Web-based software engineering?

(关于基于 Web 的软件工程，哪项陈述是错误的？)

- A. Software reuse is the dominant approach.
- B. The systems should be developed and delivered incrementally.
- C. The fundamental ideas of conventional software engineering no longer apply to web-based software.
- D. Software engineering techniques that deal with scale and complexity are relevant.
- **答案：** C
- **知识点：** Web-based Software Engineering (基于 Web 的软件工程)
 - 软件复用 (Reuse) 是主流方法 (A 对)。
 - 应采用增量式开发 (Incremental) (B 对)。
 - 传统软件工程的基本思想 (如项目管理、配置管理等) 仍然适用 (apply) 于 Web 软件 (C 错)。
 - 处理规模和复杂性的技术是相关的 (D 对)。

- **解析：** Web 开发虽然有其特殊性，但依然遵循软件工程的基本原则，所以 C 说“不再适用”是错误的。

12. 填空题

【第1章】 The textbook lists some behaviors that are not bound by laws but by professional responsibility, these are: confidentiality, [填空 1], intellectual property rights, and computer misuse.

(教材列出了一些不受法律约束但受职业责任约束的行为：保密、____、知识产权和计算机滥用。)

- **答案：** competence / 工作能力 / 能力
- **知识点：** Software Engineering Ethics (软件工程道德)
 - 四大准则：Confidentiality (保密), Competence (能力), Intellectual property rights (知识产权), Computer misuse (计算机滥用)。
- **解析：** Competence 指工程师不应接受超出其能力范围的工作，也不应虚假陈述自己的技能水平。

13. 单选题

【第1章】 Which application type does the case "wilderness weather station" in the textbook belong to?

(教材中的“野外气象站”案例属于哪种应用类型？)

- A. Data collection systems
- B. Systems for modeling and simulation
- C. Entertainment systems
- D. Batch processing systems
- **答案：** A
- **知识点：** Case Studies (案例研究) / Application Types (应用类型)
 - Wilderness weather station (野外气象站)：是一个传感器基础的 Data collection system (数据收集系统)，负责从远程区域收集气象数据。
- **解析：** 该系统通过传感器收集温度、风速等数据，属于典型的数据收集系统。

14. 填空题

When discussing processes, we usually talk about the activities in these processes such as specifying a data model, designing a user interface, etc. and the [填空 1] of these activities.

(讨论过程时，我们通常讨论过程中的活动 (如定义数据模型、设计 UI 等) 以及这些活动的____。)

- **答案：** ordering / 次序 / 顺序

- **知识点: Software Process Descriptions (软件过程描述)**
 - 过程包括: Activities (活动) 和 Ordering (次序)。
- **解析:** 过程不仅仅是一堆活动, 更重要的是这些活动执行的先后顺序和依赖关系。

15. 多选题

【第2章】 Apart from activities, software process descriptions may also include:

(除了活动之外, 软件过程描述还可以包括:)

- A. Pre-conditions
- B. Products
- C. Roles
- D. post-conditions
- **答案:** A, B, C, D (全选)
- **知识点: Software Process Descriptions (软件过程描述)**
 - 描述过程时还包括: Products (产物/结果), Roles (角色/谁来做), Pre- and post-conditions (前置和后置条件)。
- **解析:** 一个完整的过程定义需要明确输入条件、输出结果、谁负责以及满足什么条件才能开始或结束。

16. 填空题

【第2章】 The textbook introduced three software process models: the waterfall model, incremental development, and [填空1].

(教材介绍了三种软件过程模型: 瀑布模型、增量式开发和____。)

- **答案:** Integration and configuration / integration and configuration / 集成和配置 / 集成与配置
- **知识点: Software Process Models (软件过程模型)**
 - 1. The waterfall model (瀑布模型)
 2. Incremental development (增量式开发)
 3. Integration and configuration (集成和配置)
- **解析:** 这是第三种通用模型, 基于软件复用 (Reuse-oriented)。

17. 多选题

【第2章】 What situations is appropriate for using waterfall model?

(什么情况下适合使用瀑布模型?)

- A. When the requirements are well-understood.
- B. Changes will be fairly limited during the design process.
- C. Business systems which have unstable requirements.
- D. Large engineering projects where a system is developed at several sites.
- **答案:** A, B, D
- **知识点: The Waterfall Model (瀑布模型)**
 - 适用场景:
 - Requirements are well-understood (需求被很好地理解) -> A
 - Changes limited (变更有限) -> B
 - Large engineering projects (大型工程项目, 便于多地协调) -> D
 - 不适用: Unstable requirements (不稳定的需求) -> C
- **解析:** 瀑布模型僵化, 不适合需求不稳定的系统 (如大多数商业系统)。

18. 多选题

【第2章】 What are the benefits of using incremental development?

(使用增量式开发有什么好处?)

- A. The cost of implementing requirements changes is reduced.
- B. It is easier to get customer feedback.
- C. More rapid delivery and deployment of useful software.
- D. The process is not visible.
- **答案:** A, B, C
- **知识点: Incremental Development Benefits (增量式开发的优点)**
 - Cost of change is reduced (降低变更成本) -> A
 - Easier to get customer feedback (易于获得反馈) -> B
 - Rapid delivery (快速交付) -> C
 - **注意:** D (Process is not visible) 是它的缺点, 不是优点。
- **解析:** 增量开发的核心优势就是灵活应对变化和快速交付价值。

19. 单选题

【第2章】 Which statement is false about "Integration and configuration" model?

(关于“集成与配置”模型, 哪项陈述是错误的?)

- A. The user's requirements are fully satisfied and never compromised.
- B. Based on software reuse where systems are integrated from existing components.
- C. Rely on a base of reusable software components and an integrating framework.
- D. Is now the standard approach for building many types of business system.
- **答案:** A
- **知识点: Integration and Configuration (集成与配置)**
 - **缺点:** Requirements compromises are inevitable (需求妥协是不可避免的)。因为你使用的是现成的组件 (COTS), 系统可能无法完全满足用户的特定需求。
- **解析:** 既然是复用现成的, 用户就得迁就软件, 不可能像定制开发那样完全满足需求, 所以 A 是错的。

20. 单选题

【第2章】 Which statement is false about "software specification"?

(关于“软件规格说明”, 哪项陈述是错误的?)

- A. It is the process of understanding and defining what services are required from the system and identifying the constraints on the system's operation and development.
- B. Requirements are usually presented at two levels of detail: System developers need a high-level statement of the requirements; End-users and customers need a more detailed system specification.
- C. Mistakes made at this stage inevitably lead to later problems in the system design and implementation.
- D. It aims to produce an agreed requirements document that specifies a system satisfying stakeholder requirements.
- **答案:** B
- **知识点: Software Specification (软件规格说明)**
 - 需求通常分为两个层次:
 - End-users/Customers need a high-level statement (用户需要高层级/概要的需求)。

- **System developers need a detailed system specification** (开发者需要详细的规格说明)。

- **解析:** 选项 B 把两者的需求弄反了。开发者需要细节来写代码, 用户只需要知道系统大体做什么。

21. 单选题

【第2章】 In the software specification process, the activities of analysis, definition, and specification are:

(在软件规格说明过程中, 分析、定义和规格说明这些活动是:)

- A. separated
- B. parallel
- C. interleaved
- D. concurrent
- **答案:** C
- **知识点:** Requirements Engineering Process (需求工程过程)
 - 这些活动不是严格分离的, 而是 **Interleaved** (交织的)。在做定义时可能需要分析, 写规格说明时可能发现新问题又回去分析。
- **解析:** 实际上, 需求工程是一个迭代和交织的过程。

22. 单选题

【第2章】 What is the outcome of the "design and implementation" process?

(“设计与实现”过程的产出是什么?)

- A. An executable system
- B. A system specification
- C. A high-level system design
- D. A validation report
- **答案:** A
- **知识点:** Design and Implementation (设计与实现)
 - 该过程的目的是将系统规格说明转换为 **An executable system** (一个可执行的系统)。
- **解析:** 设计是规划结构, 实现是写代码, 最终结果就是能跑的软件。

23. 多选题

【第2章】 Which activities belong to "design activities"?

(哪些活动属于“设计活动”?)

- A. Architectural design
- B. Database design
- C. Interface design

- D. Component selection and design
- **答案:** A, B, C, D (全选)
- **知识点:** Design Activities (设计活动)
 - Architectural design (体系结构设计)
 - Database design (数据库设计)
 - Interface design (接口设计)
 - Component selection and design (组件选择与设计)
- **解析:** 这些都是将需求转化为具体实现方案所必需的设计步骤。

24. 填空题

【第2章】 [填空 1] is the activity of finding program faults and correcting these faults.

(____是发现程序故障并纠正这些故障的活动。)

- **答案:** Debugging / debugging / 调试
- **知识点:** System Implementation (系统实现)
 - **Debugging (调试):** 定位并修复 bug。
 - **区别:** Testing (测试) 是为了发现 bug 的存在, 而 Debugging 是为了定位并修复它。
- **解析:** 定义题。找错改错就是调试。

25. 填空题

【第2章】 Verification and validation (V & V) involves checking and review processes and system testing, among which [填空 1] is the most commonly used V & V activity.

(验证和确认 (V&V) 包括检查、评审过程和系统测试, 其中____是最常用的 V&V 活动。)

- **答案:** testing / 测试 / system testing / 系统测试
- **知识点:** Software Validation (软件确认)
 - **Testing (测试)** 是最常用的 V&V 活动。它涉及使用测试用例执行系统。
- **解析:** 虽然评审 (Review) 也很重要, 但运行代码进行测试是验证软件最直接、最普遍的方法。

26. 填空题

【第2章】 The three-stage testing process includes: component testing, [填空 1], and customer testing.

(三阶段测试过程包括: 组件测试、____和客户测试。)

- **答案:** System testing / system testing / 系统测试
- **知识点:** Stages of Testing (测试阶段)
 1. Component testing (组件测试)
 2. **System testing (系统测试):** 测试集成后的整个系

统。

3. Customer testing (客户测试)

- **解析:** 组件组装成系统, 因此中间必然是系统测试。

27. 单选题

【第2章】 In the so-called beta testing (β testing), who try out and comment on the software?

(在所谓的 Beta 测试中, 谁来试用并评价软件?)

- A. the developer
- B. the customer
- C. all stakeholders
- D. selected users
- **答案:** D
- **知识点:** Customer Testing (客户测试)
 - 对于通用软件产品, 客户测试通常称为 Beta Testing, 由 **Selected users** (选定的潜在用户) 进行。
- **解析:** Alpha 测试通常在开发者内部进行, Beta 测试则是发给一部分外部真实用户使用。

28. 判断题

【第2章】 Very few software systems are completely new systems, and it makes much more sense to see development and maintenance as two separate processes.

(很少有软件系统是全新的, 将开发和维护看作两个独立的过程更有意义。)

- **答案:** False (✗)
- **知识点:** Software Evolution (软件演化)
 - 将开发和维护看作 **A continuum** (一个连续体/整体) 更有意义, 而不是两个独立的过程 (Separate processes)。
- **解析:** 因为软件在生命周期中不断演化, 开发完第一个版本后紧接着就是基于新需求的修改 (即维护/演化), 两者界限模糊。

29. 填空题

【第2章】 The two methods used to reduce the costs of rework are: change anticipation and [填空 1].

(用于降低返工成本的两种方法是: 变更预见和____。)

- **答案:** Change tolerance / change tolerance / 变化容忍 / 变更容忍
- **知识点:** Reducing the Costs of Rework (降低返工成本)
 - **Change anticipation (变更预见):** 如 Prototyping。

- **Change tolerance (变更容忍)**: 如 Incremental delivery。
- **解析**: 要么提前猜到会变（预见），要么把系统设计成容易改（容忍）。

30. 填空题

【第2章】 There are two approaches to coping with changing requirements: system prototyping and [填空 1].
(应对需求变更的两种方法是：系统原型和____。)

- **答案**: Incremental delivery / incremental delivery / 增量式交付
- **知识点**: Coping with Changing Requirements (应对需求变更)
 - System prototyping (系统原型) -> 对应 Change anticipation。
 - Incremental delivery (增量式交付) -> 对应 Change tolerance。
- **解析**: 同上题，这是具体实施的两种技术手段。

31. 填空题

【第2章】 The technique used to improve the structure and organization of a program and thus preventing it from degrading, is called [填空 1].
(用于改进程序结构和组织，从而防止其退化的技术被称为____。)

- **答案**: Refactoring / refactoring / 重构
- **知识点**: Software Evolution / Agile (软件演化/敏捷)
 - Refactoring (重构): 在不改变代码外部行为的前提下，改善其内部结构。
- **解析**: 这是敏捷开发（特别是 XP）中的核心实践，用于对抗“软件腐烂”。

32. 判断题

【第2章】 For incremental delivery, the highest priority requirements are included in early increments and tend to receive the most testing.
(对于增量式交付，最高优先级的需求被包含在早期增量中，并且往往接受最多的测试。)

- **答案**: True (✓)
- **知识点**: Incremental Delivery Advantages (增量式交付的优点)
 - 高优先级需求在早期开发，意味着它们在后续每个增量的回归测试中都会被测到，因此接受了最多的

测试。

- **解析**: 最早做出来的功能，被测试的次数最多（因为每次发新版都要测旧功能），所以最可靠。

33. 单选题

【第2章】 Incremental delivery is not suited for the situation that
(增量式交付不适用于以下哪种情况？)

- A. The new system is intended to replace an existing system.
- B. Customers are anxious to gain value.
- C. It is easy to incorporate changes into the system, and early increments act as a prototype to help elicit requirements for later increments.
- D. The requirements is not clear initially.
- **答案**: A
- **知识点**: Incremental Delivery Problems (增量式交付的问题)
 - 如果新系统是用来 Replace an existing system (替换现有系统)，用户通常需要新系统具备旧系统的所有功能才能开始工作，分批交付会导致用户无法完成完整的工作流。
- **解析**: 替换旧系统时，通常需要“一次性”拥有完整功能，否则新系统无法完全替代旧系统，导致工作中断。

34. 填空题

【第2章】 There are two approaches to process improvement: the process maturity approach and the [填空 1] approach.
(过程改进有两种方法：过程成熟度方法和____方法。)

- **答案**: agile / 敏捷
- **知识点**: Process Improvement Approaches (过程改进方法)
 - Process maturity approach (成熟度方法): 强调规范、管理 (CMMI)。
 - Agile approach (敏捷方法): 强调迭代、减少开销。
- **解析**: 这是两种截然不同的改进哲学，一种向左（更严谨），一种向右（更灵活）。

35. 填空题

【第2章】 The general "process improvement cycle" underlying the process maturity approach is a cyclical process of three activities: measure, [填空 1] and change.

(过程成熟度方法背后的通用“过程改进循环”是一个包含三个活动的循环过程：测量、____和变更。)

- **答案**: analyze / analysis / 分析
- **知识点**: Process Improvement Cycle (过程改进循环)
 - Measure (测量) -> Analyze (分析) -> Change (变更)。
- **解析**: 先收集数据，再分析瓶颈，最后实施变更。

36. 填空题

【第2章】 Software Engineering Institute (SEI) proposed five levels of process maturity: Initial, [填空 1], Defined, Quantitatively managed, Optimizing.

(软件工程研究所 (SEI) 提出了五个过程成熟度等级：初始级、____、被定义级、定量管理级、优化级。)

- **答案**: Managed / managed / 被管理级 / 被管理 / 被管理的
- **知识点**: SEI Capability Maturity Model (SEI CMMI)
 - Level 1: Initial
 - Level 2: Managed (项目级管理)
 - Level 3: Defined (组织级标准)
 - Level 4: Quantitatively managed
 - Level 5: Optimizing
- **解析**: Level 2 关注项目层面的基本管理控制。

软件工程期末复习资料 (第四章 - 第五章)

第一部分：重点知识点整理

【第4章】需求工程 (Requirements Engineering)

1. 什么是需求工程？

- **定义 (Definition)**: 需求工程是发现、分析、记录和检查客户对系统要求的服务 (Services) 以及系统运行和开发的约束 (Constraints) 的过程。
- **核心公式**: RE = 所需服务 (Services) + 约束 (Constraints)。

2. 需求的类型 (Types of Requirement)

需求通常分为两个层次：

- **用户需求 (User Requirements):**
 - 形式: 使用自然语言 (Natural Language) 和图表编写。
 - 内容: 描述系统期望提供的服务以及操作约束。
 - 读者对象: 客户经理、系统最终用户、客户工程师、承包商经理、系统架构师。
- **系统需求 (System Requirements):**
 - 形式: 详细文档。
 - 内容: 详细描述系统的功能、服务和操作约束。它是系统买方和开发者之间合同的一部分。
 - 读者对象: 系统最终用户、客户工程师、系统架构师、软件开发者。

3. 系统干系人 (System Stakeholders)

- **定义:** 受系统影响 (Affected by the system) 并对系统有合法利益 (Legitimate interest) 的任何人。
- **范围:** 从最终用户 (End-users) 到经理 (Managers), 再到外部干系人 (如监管机构 Regulators) 。

4. 功能需求与非功能需求

- **功能需求 (Functional Requirements):**
 - 描述系统**应该做什么** (What the system should do)。
 - 说明系统应提供的服务、对特定输入的反应以及在特定情况下的行为。
 - 也可以明确声明系统**不应该做什么**。
- **非功能需求 (Non-functional Requirements, NFR):**
 - 对系统提供的服务或功能的**约束** (Constraints)。
 - 包括时间约束、开发过程约束、标准等。
 - 通常适用于**整个系统** (System as a whole), 而非单个功能。
 - **关键性:** 非功能需求往往比单个功能需求更关键。如果未能满足 (如可靠性、安全性), 可能导致整个系统不可用。

5. 非功能需求的分类

- **产品需求 (Product Requirements):**
 - 刻画或约束软件的运行时行为 (Runtime behavior)。
 - 例如: 性能 (Performance)、可靠性 (Reliability)、效率 (Efficiency)、可用性 (Usability)、安全性 (Security)。
- **组织需求 (Organizational Requirements):**
 - 源于客户和开发者组织的政策和程序。

- 例如: 交付标准、实施要求、开发标准。

- **外部需求 (External Requirements):**
 - 源于系统及其开发过程之外的因素。
 - 例如: 互操作性 (Interoperability)、法律 (Legislative)、道德 (Ethical) 需求。

6. 需求的完备性与一致性

- **完备性 (Completeness):** 用户所需的所有服务和信息都应该被定义。
- **一致性 (Consistency):** 需求之间不应存在矛盾 (Contradictory)。
- **注:** 在大型复杂系统中, 实际上很难同时达到完美的完备性和一致性。

7. 需求工程过程 (Requirements Engineering Processes)

包含三个关键活动 (实际上是迭代交织的):

1. **需求抽取与分析 (Elicitation and Analysis):** 发现需求。
2. **需求规格说明 (Specification):** 将需求转化为标准形式。
3. **需求确认 (Validation):** 检查需求是否定义了客户想要的系统。

8. 需求抽取技术 (Elicitation Techniques)

- **访谈 (Interviewing):**
 - **封闭式访谈 (Closed interviews):** 回答预定义的问题。
 - **开放式访谈 (Open interviews):** 没有预定义议程 (No predefined agenda), 探索各种问题。
- **人种志/人种学调查 (Ethnography):**
 - 观察人们的实际工作方式。
 - 特长: 特别适合发现**隐含需求 (Implicit system requirements)** 和源于合作的需求。
 - 缺点: 不适合发现新的产品创新或更广泛的组织/领域需求。
- **故事与场景 (Stories and Scenarios):**
 - **故事 (Stories):** 叙事性文本, 高层描述, 适合设定“宏观图景” (Big picture)。
 - **场景 (Scenarios):** 结构化的描述, 包含具体的交互细节 (开始状态、正常流程、异常处理、结束状态等)。

9. 需求规格说明 (Requirements Specification)

- **自然语言 (Natural Language):** 最常用, 通用性强, 但可

能存在歧义 (Ambiguous)。

- **结构化语言 (Structured specifications):** 使用标准模板或表单, 限制自由文本, 提高一致性。
- **表格规格说明 (Tabular specification):** 适用于描述面对多种选择情况下的动作 (如: 如果 A 则 X, 如果 B 则 Y) 。
- **撰写指南:**
 - **Mandatory (强制性)** 需求通常用 “shall”。
 - **Desirable (期望性)** 需求通常用 “should”。

10. 需求确认 (Requirements Validation)

- **目的:** 检查需求是否定义了客户真正想要的系统。
- **检查内容:**
 - **有效性 (Validity):** 是否符合真实需求?
 - **一致性 (Consistency):** 是否有冲突?
 - **完备性 (Completeness):** 是否包含所有功能和约束?
 - **现实性 (Realism):** 预算和技术是否可行?
 - **可验证性 (Verifiability):** 需求是否可以被测试?
- **技术:** 需求评审 (Reviews)、原型 (Prototyping)、测试用例生成 (Test-case generation)。

11. 需求变更管理 (Requirements Management)

- **过程:**
 1. **问题分析与变更规格说明 (Problem analysis and change specification)。**
 2. **变更分析与成本计算 (Change analysis and costing)。**
 3. **变更实现 (Change implementation)。**
- **敏捷开发:** 通常不通过正式的变更管理过程, 而是设计用来应对变更。

【第 5 章】系统建模 (System Modeling)

1. 什么是系统建模?

- **定义:** 开发系统抽象模型 (Abstract models) 的过程, 每个模型从不同的视角 (Perspective) 展示系统。
- 通常基于 UML (Unified Modeling Language)。

2. 系统视角 (System Perspectives)

- **外部视角 (External perspective):** 对系统的上下文或环境建模。
- **交互视角 (Interaction perspective):** 对系统与环境或组件之间的交互建模。

- **结构视角 (Structural perspective)**: 对系统的组织或数据结构建模。
- **行为视角 (Behavioral perspective)**: 对系统的动态行为及对事件的响应建模。

3. UML 图表类型 (UML Diagram Types)

- **活动图 (Activity diagrams)**: 显示过程或数据处理中的活动 (流程)。
- **用例图 (Use case diagrams)**: 显示系统与环境之间的交互。
- **顺序图 (Sequence diagrams)**: 显示参与者、系统及组件之间的交互顺序。
- **类图 (Class diagrams)**: 显示系统中的对象类及其关联。
- **状态图 (State diagrams)**: 显示系统如何对内部和外部事件做出反应。

4. 上下文模型 (Context Models)

- 用于在早期阶段确定**系统边界 (System Boundaries)** (即什么在系统内, 什么在系统外)。
- 显示系统与环境中的其他自动化系统的关系。

5. 交互模型 (Interaction Models)

- **用例建模 (Use Case Modeling)**:
 - 主要用于需求抽取。
 - **参与者 (Actors)**: 可以是人或其他系统, 用火柴人表示。
 - **用例 (Use Case)**: 表示一个离散的任务, 用椭圆表示。
- **顺序图 (Sequence Diagrams)**:
 - 对参与者、对象之间的交互建模。
 - 垂直虚线表示对象的**生命线 (Lifeline)**。
 - 箭头表示对象间的调用或消息传递。
 - alt 框表示选择结构 (如 if-else)。

6. 结构模型 (Structural Models)

- **类图 (Class Diagrams)**:
 - **类 (Class)**: 矩形框, 包含类名、属性、操作。
 - **泛化 (Generalization)**: 即继承 (Inheritance), 用空心三角箭头指向父类。表示 "is-a" 关系。子类继承父类的属性和操作。
 - **聚集 (Aggregation)**: 表示 "part-of" 关系 (整体与部分), 用空心菱形指向整体。
 - **关联 (Association)**: 类之间的连接, 可标注数量 (如

1.* 表示 1 到多)。

7. 行为模型 (Behavioral Models)

- **数据驱动建模 (Data-driven modeling)**:
 - 显示输入数据如何被处理并生成输出。
 - 常用 **活动图 (Activity diagrams)** 表示。
- **事件驱动建模 (Event-driven modeling)**:
 - 显示系统如何响应外部和内部事件 (Events)。
 - 基于假设: 系统有有限个状态, 事件触发状态转换。
 - 常用 **状态图 (State diagrams)** 表示。

8. 模型驱动工程 (Model-Driven Engineering, MDE)

- **核心思想**: 模型 (Models) 而非程序是开发过程的主要输出。程序自动从模型生成。
- **MDA (Model-Driven Architecture)**: MDE 的一种具体实现, 关注设计和实现阶段。
- **三种模型 (MDA)**:
 1. **CIM (Computation Independent Model)**: 计算无关模型 (领域模型)。
 2. **PIM (Platform Independent Model)**: 平台无关模型 (UML 模型, 不涉及具体实现细节)。
 3. **PSM (Platform Specific Model)**: 平台相关模型 (针对特定平台如 J2EE, .NET)。
- **转换**: 通过 **转换器 (Translator)** 工具将 PIM 转换为 PSM, 再生成代码。
- **注意**: 从 CIM 到 PIM 的转换通常需要人工干预。

第二部分: 习题与解析 (第二次作业 第 4-5 章)

试卷信息: 共 37 题, 满分 40 分

1. 填空题

【第 4 章】 Requirements engineering is the process of finding out, analyzing, documenting and checking the services the customer requires from a system, and [填空 1] under which it operates and is developed.

- **答案**: constraints / constraint / 约束
- **知识点解析**:
 - **定义 (Definition)**: 软件工程是一门工程学科, 它关注软件生产的所有方面。
 - **需求工程 (Requirements Engineering)**: 它是发现、分析、记录和检查客户对系统要求的服务 (Services)

以及系统运行和开发的**约束 (Constraints)** 的过程。

- 这个定义强调了 RE 的两个核心产出: 所需的服务 (What the system should do) 和 约束 (Constraints)。

2. 填空题

【第 4 章】 Requirements is distinguished between high-level abstract requirements and detailed description of what should do. The former is called user requirements, and the later is called [填空 1] requirements.

- **答案**: system / 系统
- **知识点解析**:
 - **需求的类型 (Types of requirement)**:
 - **用户需求 (User requirements)**: 高层抽象需求, 使用自然语言和图表描述系统期望提供的服务。它是给客户经理和最终用户看的。
 - **系统需求 (System requirements)**: 对系统功能、服务和操作约束的详细描述 (Detailed description), 定义了具体要实现的内容。它是给开发者和架构师看的。

3. 单选题

【第 4 章】 Which option is the most appropriate concept for the statement below? "Any person who is affected by the system in some way and so who has a legitimate interest."

- A. users
- B. stakeholders
- C. managers
- D. regulators

● **答案**: B

- **知识点解析**:
 - **系统干系人 (System Stakeholders)**: 定义为受系统影响 (Affected by the system) 并对系统有合法利益 (Legitimate interest) 的任何人。
 - **范围**: Stakeholders 是一个广泛的概念, 它包含了 A、C、D 选项中的所有角色 (用户、经理、监管者), 以及其他如开发者、维护人员等。题目描述的是这个群体的统称。

4. 单选题

【第 4 章】 Which of the statements belongs to functional requirements?

- A. Reliability of the system
- B. Response time of the system
- C. Services the system should provide
- D. Security of the system

● 答案: C

● 知识点解析:

- 功能需求 (Functional requirements): 描述系统应该做什么, 即系统应提供的服务 (Services), 以及系统对特定输入的反应。
- 非功能需求 (Non-functional requirements): 描述系统的属性和约束。A (可靠性)、B (响应时间)、D (安全性) 都属于非功能需求。

5. 判断题

【第4章】 The statement below defines the consistency of requirements. "All services and information required by the user should be defined."

● 答案: False (✗)

● 知识点解析:

- 完备性 (Completeness): 用户所需的所有服务和信息都应被定义 ("All services... should be defined")。
- 一致性 (Consistency): 需求之间不应存在矛盾。
- 题目中的描述是完备性的定义, 而不是一致性。

6. 判断题

【第4章】 At any time, non-functional requirements are less critical than individual functional requirements.

● 答案: False (✗)

● 知识点解析:

- 非功能需求的重要性: 非功能需求往往比单个功能需求更关键 (More critical)。
- 如果未能满足非功能需求 (如安全性、可靠性), 可能导致整个系统不可用, 而单个功能失效可能只会影响部分操作。例如, 一个不安全的银行系统是完全无法使用的, 无论其转账功能多么完美。

7. 判断题

【第4章】 A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements.

● 答案: True (✓)

● 知识点解析:

- 非功能需求的实现: 一个非功能需求 (如安全性要求) 可能会产生多个相关的功能需求来定义新的系统服务。
- 例如: 为了满足“系统必须安全”这一 NFR, 可能需要增加“用户登录”、“权限管理”、“操作日志记录”等多个功能需求。

8. 单选题

【第4章】 What is wrong with the non-functional requirement below? "The system should be easy to use by medical staff and should be organized in such a way that user errors are minimized."

- A. There exist grammatical problems.
- B. The end-user is not mentioned.
- C. There exist contradictions in the statement.
- D. The statement is impossible to verify.

● 答案: D

● 知识点解析:

- 可验证性 (Verifiability): 非功能需求应定量编写以便客观测试。
- 目标 (Goal) vs 需求: 题目中的描述是一个“目标” (如 “easy to use”), 这是主观的, 无法通过测试进行客观验证 (Impossible to verify)。
- 应转化为可验证的需求, 例如: “医务人员在培训 2 小时后应能使用所有功能, 且平均错误率不超过...”

9. 多选题

【第4章】 Which options are commonly used metrics for specifying non-functional requirements?

- A. Speed
- B. Ease of use
- C. Reliability
- D. Robustness

● 答案: A, B, C, D

● 知识点解析:

- 非功能需求的度量 (Metrics): 为了使非功能需求可验证, 需要使用度量标准。
- 常见的度量属性包括:
 - 速度 (Speed): 处理事务数/秒、响应时间、屏

幕刷新时间。

- 易用性 (Ease of use): 培训时间、帮助帧数。
- 可靠性 (Reliability): 平均故障间隔时间 (MTTF)、可用性概率。
- 鲁棒性 (Robustness): 故障重启时间、故障导致的数据损坏概率。

10. 填空题

【第4章】 Requirements engineering involves three key activities: elicitation and analysis, [填空 1] and validation.

● 答案: specification / 规格说明

● 知识点解析:

- 需求工程过程 (RE Processes) 包含三个关键活动 (迭代进行):
 1. 需求抽取与分析 (Elicitation and analysis): 发现需求。
 2. 需求规格说明 (Specification): 将需求转化为标准形式 (文档)。
 3. 需求确认 (Validation): 检查需求是否正确定义了系统。

11. 填空题

【第4章】 There are two fundamental approaches to requirements elicitation: interviewing, observation or [填空 1].

● 答案: ethnography / 人种志 / 人种学调查

● 知识点解析:

- 需求抽取技术: 主要有两种基本途径:
 1. 访谈 (Interviewing): 与人交谈了解他们做什么。
 2. 观察或人种志 (Observation or ethnography): 通过观察人们的实际工作 (immersing in the working environment) 来了解操作流程。

12. 判断题

【第4章】 The difference between stories and scenarios is in the ways that descriptions are structured and in the level of detail presented, where scenarios are effective in setting out the "big picture".

● 答案: False (✗)

● 知识点解析:

- 故事 (Stories): 叙事性文本, 呈现系统使用的高层描述。Stories are effective in setting out the "big

picture" (宏观图景)。

- **场景 (Scenarios)**: 通常是结构化的, 包含具体的细节 (如输入、输出、异常处理)。
- 题目说反了, Story 才是适合描述 Big picture 的。

13. 判断题

【第4章】 Mandatory requirements are usually written using "shall", while desirable requirements are written using "should".

- **答案**: True (✓)
- **知识点解析**:
 - **需求撰写指南**: 为了清晰区分需求的优先级, 标准做法是:
 - **强制性 (Mandatory)** 需求: 必须实现, 使用 "shall".
 - **期望性 (Desirable)** 需求: 最好实现, 使用 "should".

14. 判断题

For open interviews, there is no predefined agenda.

- **答案**: True (✓)
- **知识点解析**:
 - **访谈类型**:
 - **封闭式访谈 (Closed interviews)**: 干系人回答一组预定义的问题。
 - **开放式访谈 (Open interviews)**: 没有预定义的议程 (No predefined agenda), 需求工程团队与干系人广泛探讨各种问题。

15. 判断题

When there are a number of possible alternative situations and you need to describe the actions to be taken for each of these, form-based specifications are particularly useful.

- **答案**: False (✗)
- **知识点解析**:
 - **表格规格说明 (Tabular specification)**: 当存在多种可能的替代情况 (alternative situations) 且需要描述每种情况下的动作时, 表格 (Tables) 特别有用。
 - 基于表单的规格说明 (Form-based specifications) 虽然也是结构化的, 但通常用于定义单个功能的输入输出和前置/后置条件, 处理复杂的条件分支不如表格直观。

16. 单选题

【第4章】 Which option is not an actor in the use case diagram below? (Generate report, Nurse, Manager, Doctor)

- A. Generate report
- B. Nurse
- C. Manager
- D. Doctor
- **答案**: A
- **知识点解析**:
 - **用例图 (Use case diagrams)** 元素:
 - **参与者 (Actor)**: 用火柴人表示, 代表与系统交互的人或外部系统 (如 Nurse, Manager, Doctor)。
 - **用例 (Use case)**: 用椭圆表示, 代表系统提供的功能或任务。
 - "Generate report" 在图中应为一个椭圆, 代表一个用例 (Use case), 而不是参与者。

17. 单选题

【第4章】 In what document will the errors found lead to the most extensive rework?

- A. design document
- B. coding document
- C. validating document
- D. requirements document
- **答案**: D
- **知识点解析**:
 - **需求确认的重要性**: 如果在系统开发后期或投入使用后才发现需求文档 (Requirements document) 中的错误, 会导致大量的返工 (Extensive rework)。
 - 因为修复需求错误意味着系统设计 (Design) 和实现 (Implementation) 都必须随之变更, 然后还需要重新测试。

18. 填空题

【第4章】 There are three requirements validation techniques: requirements reviews, [填空 1], and test-case generation.

- **答案**: prototyping / 原型
- **知识点解析**:
 - **需求确认技术**包括:

1. **需求评审 (Requirements reviews)**: 系统地分析需求文档。
2. **原型 (Prototyping)**: 开发可执行模型演示给用户, 看是否符合需求。
3. **测试用例生成 (Test-case generation)**: 如果在编码前能生成测试, 说明需求是可测试的。

19. 判断题

【第4章】 For agile development processes, a requirement change does not go through a formal change management process.

- **答案**: True (✓)
- **知识点解析**:
 - **敏捷开发 (Agile development)**: 敏捷方法的设计初衷就是为了应对需求变更。
 - 在敏捷过程中, 变更通常不通过正式的变更管理过程 (Does not go through a formal change management process), 而是通过迭代计划直接处理。

20. 填空题

【第4章】 Requirements change management will go through three stages: problem analysis and change specification, change analysis and costing, and [填空 1].

- **答案**: change implementation / 变更实现
- **知识点解析**:
 - **需求变更管理过程**的三个主要阶段:
 1. 问题分析与变更规格说明。
 2. 变更分析与成本计算。
 3. **变更实现 (Change implementation)**: 修改需求文档, 并在必要时修改系统设计和实现。

21. 填空题

【第5章】 System modeling is the process of developing abstract models of a system, with each model presenting a different view or [填空 1] of that system.

- **答案**: perspective / 视角
- **知识点解析**:
 - **系统建模 (System modeling)**: 开发系统抽象模型的过程。
 - 由于单个模型无法覆盖系统的所有方面, 因此每个模型只展示系统的不同视角 (Perspective) (如外部

视角、交互视角、结构视角、行为视角)。

22. 多选题

【第5章】 You may develop different models to represent the system from different perspectives, these perspectives include:

- A. An external perspective
- B. An interaction perspective
- C. A structural perspective
- D. A behavioral perspective

● 答案: A, B, C, D

● 知识点解析:

- 系统视角 (System perspectives) 包括:
 - 外部视角 (External): 对上下文或环境建模。
 - 交互视角 (Interaction): 对系统与环境或组件间的交互建模。
 - 结构视角 (Structural): 对系统的组织或数据结构建模。
 - 行为视角 (Behavioral): 对系统的动态行为建模。

23. 单选题

【第5章】 Which option does not belong to UML diagrams?

- A. Block diagrams
- B. Activity diagrams
- C. Use case diagrams
- D. Sequence diagrams

● 答案: A

● 知识点解析:

- UML 图表: UML 包含 13 种标准图表, 如活动图 (Activity)、用例图 (Use case)、顺序图 (Sequence)、类图 (Class)、状态图 (State) 等。
- 框图 (Block diagrams): 虽然常用于工程中表示结构, 但它不是 UML 标准定义的图表类型之一 (UML 中对应的是组件图或部署图, 或者 SysML 中的 Block Definition Diagram)。

24. 填空题

【第5章】 At an early stage in the specification of a system, you should decide on the system [填空 1], that is, on what is and is not part of the system being developed.

● 答案: boundaries / boundary / 边界

● 知识点解析:

- 上下文模型 (Context models): 用于在早期阶段确定系统边界 (System boundaries)。
- 决策内容: 确定哪些属于被开发的系统 (System being developed), 哪些属于外部环境。

25. 单选题

【第5章】 Which statement is false about the UML activity diagram below?

- A. The filled-circle is the start...
- B. Once the "Confirm detention decision" activity ends, both... can start and execute in parallel.
- C. As long as one of the three activities ("Inform social care" etc.) is completed, the activity diagram ends.
- D. Only when all the three activities... are completed, the activity diagram ends.

● 答案: C

● 知识点解析:

- 活动图 (Activity diagrams) 中的同步条 (Solid bar):
 - 当流程从一个活动分叉 (Fork) 到多个活动时, 表示并行执行。
 - 当多个活动汇合 (Join) 到一个条时, 表示同步。规则是: 只有当汇合条的所有输入活动 (即图中的那三个通知活动) 全部完成 (All ... are completed) 后, 流程才能继续并结束。
- 因此, C 选项说"只要完成一个就结束"是错误的。

26. 多选题

【第5章】 Which options belong to interaction modeling?

- A. Activity diagrams
- B. State diagrams
- C. Use case diagrams
- D. Sequence diagrams

● 答案: C, D

● 知识点解析:

- 交互模型 (Interaction models):
 - 用例图 (Use case diagrams): 主要用于对系统与外部代理 (用户或其他系统) 之间的交互建模。
 - 顺序图 (Sequence diagrams): 用于对系统组

件之间 (或对象之间) 的交互顺序建模。

- A 是过程建模, B 是事件/行为建模。

27. 单选题

【第5章】 Which statement is false about the sequence diagram below?

- A. "ViewInfo (PID)" occurs before...
- B. The function "report..." belongs to the object "D: Men tcare-DB".
- C. Either "Error" or "Patient info" occurs according to the condition...
- D. "Error (no access)" occurs before "Patient info".

● 答案: D

● 知识点解析:

- 顺序图 (Sequence diagrams):
 - Alt 框 (Alternative): 表示选择结构 (类似 if-else)。
 - 图中的 alt 框显示: [authorization OK] 执行 Patient info, [authorization fail] 执行 Error。
 - 这两个分支是互斥的, 根据条件执行其中一个, 绝对不会出现一个发生在另一个之前 (Before) 的情况。因此 D 错误。

28. 单选题

【第5章】 Which option represents a 1-to-many relationship in the class diagram below?

- A. Condition---Patient
- B. Patient---Consultation
- C. Consultant---Patient
- D. Consultation---Medication

● 答案: C

● 知识点解析:

- 类图关联 (Association): 通过连线两端的数字 (多重性) 判断。
- 1-to-many: 一端是 1, 另一端是 * 或 1.*。
- 根据 PPT 中的类图: Consultant 端标注为 1, Patient 端标注为 1.*。这表示一个顾问医生 (Consultant) 对应多个病人 (Patient), 符合 1-to-many 关系。

29. 单选题

【第5章】 Which statement is false about the generalization

hierarchy shown below?

- A. The "Hospital doctor" also has "Phone #" attribute.
- B. The "Doctor" also has "Staff #" attribute.
- C. The "General practitioner" can call the "register()" method.
- D. The "Hospital doctor" can call the "de-register()" method.
- **答案:** B
- **知识点解析:**
 - **泛化 (Generalization/Inheritance):** 子类继承父类的属性和方法。
 - **父类:** Doctor (属性: Phone #; 方法: register)。
 - **子类:** Hospital doctor (特有属性: Staff #)。
 - **规则:** 子类拥有父类的所有属性 (所以 A 正确), 但父类 (Doctor) 不包含子类 (Hospital doctor) 特有的属性 (Staff #)。因此 B 错误。

30. 填空题

【第 5 章】 The UML provides a special type of association between classes called [填空 1], which means that one object (the whole) is composed of other objects (the parts).

- **答案:** aggregation / 聚集
- **知识点解析:**
 - **聚集 (Aggregation):** UML 中一种特殊的关联类型。
 - **含义:** 表示**整体与部分 (Whole-part)** 的关系 ("is composed of")。
 - **符号:** 在代表"整体"的类一端使用空心菱形。

31. 填空题

【第 5 章】 Behavioral models are models of the dynamic behavior of a system as it is executing... These stimuli may be either [填空 1] or events.

- **答案:** data / 数据
- **知识点解析:**
 - **行为模型 (Behavioral models):** 描述系统的动态行为。
 - 系统响应来自环境的刺激 (Stimuli)。
 - 刺激主要分为两种:
 1. **数据 (Data):** 数据驱动建模 (数据到达触发处理)。

2. 事件 (Events): 事件驱动建模 (事件发生触发状态转换)。

32. 单选题

【第 5 章】 What kind of diagram is used by the UML to support event-based modeling?

- A. sequence diagram
- B. state diagram
- C. activity diagram
- D. use case diagram
- **答案:** B
- **知识点解析:**
 - **事件驱动建模 (Event-driven modeling):** 显示系统如何响应外部和内部事件。
 - **状态图 (State diagrams):** 显示系统状态 (System states) 以及引起从一个状态转换到另一个状态的事件 (Events)。这是 UML 支持事件建模的主要工具。

33. 判断题

【第 5 章】 Topics such as model-based requirements engineering, model-based development, and model-based testing are part of MDA (Model-Driven Architecture) but are not considered in MDE (Model-Driven Engineering).

- **答案:** False (✗)
- **知识点解析:**
 - **MDE vs MDA:**
 - **MDA (Model-Driven Architecture):** 主要关注软件开发的设计和实现阶段。
 - **MDE (Model-Driven Engineering):** 关注软件工程过程的所有方面。
 - 因此, 基于模型的需求工程、测试等属于范围更广的 MDE, 而不属于主要关注实现的 MDA。题目说反了。

34. 填空题

【第 5 章】 The MDA (Model-Driven Architecture) method recommends that three types of abstract system model should be produced: CIM, [填空 1], and PSM.

- **答案:** PIM / 平台无关模型
- **知识点解析:**

- **MDA 的三种模型:**
 1. **CIM (Computation Independent Model):** 计算无关模型。
 2. **PIM (Platform Independent Model):** 平台无关模型 (描述系统操作但不涉及实现细节)。
 3. **PSM (Platform Specific Model):** 平台相关模型。

35. 填空题

【第 5 章】 By using powerful tools, system implementations can be generated... to adapt the system to some new platform, you write a model [填空 1] for that platform.

- **答案:** translator / 转换器
- **知识点解析:**
 - **MDA 的优势:** 从同一个 PIM 生成不同平台的实现。
 - **实现方式:** 为了适配新平台, 需要为该平台编写一个**模型转换器 (Model translator)**, 利用特定平台的规则和模式将 PIM 转换为 PSM。

36. 单选题

【第 5 章】 According to the MDA transformations shown in the figure below, which transformation needs human intervention?

- A. From CIM to PIM
- B. From PIM to PSM
- C. From PSM to Code
- D. All transformations need human intervention
- **答案:** A
- **知识点解析:**
 - **MDA 转换:**
 - **PIM 到 PSM、PSM 到代码:** 通常由工具自动完成。
 - **CIM 到 PIM:** 从高层领域抽象模型 (CIM) 到平台无关模型 (PIM) 的转换仍然是一个研究难题, 在实际生产中通常需要**人工干预 (Human intervention)**。

37. 判断题

【第 5 章】 There is an uneasy relationship between agile methods and MDA (Model-Driven Architecture).

- 答案: True (✓)
- 知识点解析:
 - 敏捷 vs MDA:
 - MDA: 强调广泛的前期建模 (Extensive up-front modeling)。
 - 敏捷: 强调代码优于文档, 响应变化, 很少有敏捷开发者对模型驱动工程感到适应。
 - 因此, 两者之间存在一种不和谐的关系 (Uneasy relationship)。

第一部分: 重点知识点整理

【第 6 章】体系结构设计 (Architectural Design)

1. 什么是体系结构设计?

- 定义 (Definition): 体系结构设计关注如何理解软件系统的组织方式, 以及如何设计系统的整体结构 (Overall structure)。
- 输出 (Output): 体系结构设计过程的输出是一个**体系结构模型 (Architectural model)**, 它描述了系统是如何组织成一组相互通信的构件 (Communicating components) 的。
- 关键环节: 它是设计与需求工程之间的关键环节。

2. 敏捷与体系结构 (Agility and Architecture)

- 早期设计: 通常认为, 敏捷开发过程的早期阶段应该关注设计一个整体的系统体系结构。
- 重构 (Refactoring):
 - 构件重构 (Refactoring components): 通常相对容易。
 - 体系结构重构 (Refactoring the system architecture): 非常昂贵 (Expensive), 因为可能需要修改大多数系统构件以适应体系结构的变化。

3. 体系结构抽象 (Architectural Abstraction)

- 小体系结构 (Architecture in the small): 关注单个程序 (Individual programs) 的体系结构。在这个层次上, 关注单个程序如何分解为构件。
- 大体系结构 (Architecture in the large): 关注包含其他系统、程序和程序构件的复杂企业系统 (Complex enterprise

systems) 的体系结构。这些系统通常分布在不同的计算机上。

4. 体系结构设计决策 (Architectural Design Decisions)

5. 非公式化 (No formulaic process): 没有公式化的体系结构设计过程。它是一个创造性过程, 取决于系统类型、架构师经验等。
6. 决策驱动: 最好将体系结构设计看作是一系列需要做出的决策 (Decisions), 而不是一系列活动。
7. 非功能性需求 (Non-functional requirements): 体系结构风格的选择应取决于系统的非功能性需求, 因为它们之间关系密切:
 - 性能 (Performance): 减少构件间通信, 使用大粒度构件。
 - 安全性 (Security): 使用分层体系结构, 核心资产在最内层。
 - 安全 (Safety): 安全相关操作集中在少数构件中。
 - 可用性 (Availability): 包含冗余构件 (Redundant components)。
 - 可维护性 (Maintainability): 使用细粒度、自包含的构件。

5. 体系结构视图 (Architectural Views) - 4+1 视图模型

Kruchten 提出的 4+1 视图模型 (4+1 view model) 包含四个基本视图, 并通过用例 (Use cases) 或场景联系起来:

- 逻辑视图 (Logical view): 将系统中的关键抽象显示为对象或对象类。
- 进程视图 (Process view): 显示系统在运行时 (Runtime) 如何由交互的进程组成。
- 开发视图 (Development view): 显示软件如何分解以进行开发 (对程序员有用)。
- 物理视图 (Physical view): 显示系统硬件以及软件构件如何在处理器上分布。
- +1: 相关的用例或场景 (Use cases or scenarios)。

6. 体系结构模式 (Architectural Patterns)

(1) MVC 模式 (Model-View-Controller)

- 描述: 将**演示 (Presentation)** 和**交互 (Interaction)** 与**系统数据 (System data)** 分离。

- 组成:
 - 模型 (Model): 管理系统数据和相关操作。
 - 视图 (View): 定义和管理数据如何呈现给用户。
 - 控制器 (Controller): 管理用户交互 (如按键、点击) 并传递给视图和模型。
- 适用场景: 有多种方式查看和交互数据; 未来的交互需求未知。
- 优点: 允许数据独立于表示而变化; 支持同一数据的多种表现形式。

(2) 分层体系结构 (Layered Architecture)

- 描述: 将系统组织成**层 (Layers)**, 每一层提供服务给它上面的层, 最底层代表核心服务。
- 适用场景: 在现有系统之上构建新设施; 多团队开发 (每队一层); 多级安全需求。
- 优点: 支持增量开发 (Incremental development); 只要接口不变, 可替换整个层; 可提供冗余设施提高可靠性。
- 缺点: 很难清晰分离各层; 性能可能因多层解释而受损。

(3) 仓库模式 (Repository Pattern)

- 描述: 所有数据由一个**中央仓库 (Central repository)** 管理, 所有系统构件都可访问。构件不直接交互, 而是通过仓库交互。
- 适用场景: 生成大量信息且需长期保存的系统; 数据驱动系统。
- 优点: 构件独立, 无需知道其他构件存在; 数据管理一致 (如备份)。
- 缺点: 仓库是单点故障 (Single point of failure); 通过仓库通信可能效率低。

(4) 客户机-服务器模式 (Client-Server Pattern)

- 描述: 系统作为一组**服务 (Services)** 呈现, 每个服务由单独的**服务器 (Server)** 提供。**客户机 (Clients)** 访问并使用这些服务。
- 适用场景: 共享数据库需从不同位置访问; 负载变化大。
- 优点: 服务器可分布在网络中; 通用功能可供所有客户机使用。
- 缺点: 服务是单点故障; 性能受网络影响不可预测。

(5) 管道和过滤器模式 (Pipe and Filter Pattern)

- 描述: 每个处理构件 (过滤器) 是离散的, 执行一种类型的**数据转换 (Data transformation)**。数据从一个构件流向

另一个构件（管道）。

- **适用场景：**数据处理应用（批处理或基于事务），输入分阶段处理生成输出。
- **优点：**易于理解，支持转换复用；符合业务流程结构。
- **缺点：**数据传输格式必须统一（增加解析/反解析开销）；不适合交互式系统。

【第 7 章】设计和实现 (Design and Implementation)

1. 设计与实现基础

- **定义：**软件设计和实现是软件工程过程中开发可执行软件系统 (Executable software system) 的阶段。
- **构建或购买 (Build or buy)：**对于许多类型的应用，现在可以购买现成的 (Off-the-shelf) 应用系统，并根据用户需求进行改编和剪裁 (Adapted and tailored)。

2. 面向对象设计 (Object-Oriented Design)

- 4. **对象 (Objects)：**包含**数据 (Data)** 和操作数据的**操作 (Operations)**。

5. 设计过程阶段：

- 理解并定义上下文 (Context) 和外部交互。
- 设计系统体系结构。
- 识别主要对象。
- 开发设计模型。
- 刻画接口。

3. 上下文与交互模型 (Context and Interaction Models)

- **上下文模型 (Context model)：**结构模型，展示系统环境中的其他系统。这有助于建立系统的**边界 (Boundaries)**。
- **交互模型 (Interaction model)：**动态模型，展示系统在运行时如何与环境交互（如使用**用例模型 Use case model**）。

4. UML 设计模型 (Design Models)

- **子系统模型 (Subsystem models)：**结构模型，显示对象如何逻辑分组为子系统。
- **顺序模型 (Sequence models)：**动态模型，显示对象**交互的顺序 (Sequence of interactions)**。
- **状态机模型 (State machine models)：**动态模型，显示对象如何根据事件改变**状态 (State)**。

5. 接口规范 (Interface Specification)

- **目的：**刻画构件间的接口，以便对象和子系统可以**并行设计 (Designed in parallel)**。
- **内容：**不应包含数据表示的细节，但应包含访问和更新数据的操作。
- **关系：**对象和接口之间**不是**简单的 1:1 关系（一个对象可能有多个接口，一组对象可能共用一个接口）。

6. 设计模式 (Design Patterns)

- **定义：**是对问题及其解决方案精髓的描述，是复用经验的一种方式。通常与面向对象设计相关。
- **四人帮 (Gang of Four) 定义的四个基本要素：**
 - **名称 (Name)：**有意义的引用。
 - **问题描述 (Description of the problem)：**何时应用该模式。
 - **解决方案描述 (Solution description)：**设计部分及其关系、职责（模板）。
 - **后果陈述 (Statement of the consequences)：**应用模式的结果和权衡。

常见模式举例：

- **观察者模式 (Observer Pattern)：**将对象状态的显示与对象本身分离。当对象状态改变时，所有显示自动更新。
 - 涉及对象：Subject（目标）和 Observer（观察者）。
- **门面模式 (Facade)：**整理一组相关对象的接口。
- **迭代器模式 (Iterator)：**提供访问集合元素的标准方式。
- **装饰者模式 (Decorator)：**允许在运行时扩展类的功能。

7. 实现问题 (Implementation Issues)

- **宿主机-目标机开发 (Host-target development)：**开发在宿主机 (Host system) 上进行，执行在目标机 (Target system) 上进行。

第二部分：作业解析

1. 填空题

题目：【第 6 章】 The output of the architectural design process is an architectural model that describes how the system is organized as a set of communicating

\$\$填空 1\$\$

- **答案：**components / 构件 / 组件
- **相关知识：**什么是体系结构设计？
输出 (Output)：体系结构设计过程的输出是一个体系结构模型 (Architectural model)，它描述了系统是如何组织成一组相互通信的构件 (Communicating components) 的。
- **解析：**根据第 6 章 PPT 中关于体系结构设计定义的描述，输出模型的核心组成部分就是“communicating component s”。

2. 判断题

题目：【第 6 章】 Refactoring components in response to changes is usually relatively easy. However, refactoring the system architecture is expensive because you may need to modify most system components to adapt them to the architectural changes.

- **答案：**True (✔)
- **相关知识：**敏捷与体系结构 (Agility and Architecture)
重构 (Refactoring)：构件重构通常相对容易。体系结构重构非常昂贵 (Expensive)，因为可能需要修改大多数系统构件以适应体系结构的变化。
- **解析：**题目完整复述了 PPT 中关于敏捷开发中重构难度的对比，局部构件修改容易，但整体架构修改牵一发而动全身，成本高昂。

3. 填空题

题目：【第 6 章】 You can design software architectures at two levels of abstraction: Architecture in the small, and

\$\$填空 1\$\$

- **答案：**Architecture in the large / 大体体系结构 / architecture in the large
- **相关知识：**体系结构抽象 (Architectural Abstraction)
小体系结构 (Architecture in the small)：关注单个程序。大体体系结构 (Architecture in the large)：关注包含其他系统、程序和程序构件的复杂企业系统的体系结构。
- **解析：**PPT 明确指出了体系结构设计两个抽象层次：Small（关注单个程序）和 Large（关注复杂企业系统）。

4. 判断题

题目：【第6章】 There is a formulaic architectural design process, in which the architectural design is considered as a sequence of activities.

- **答案：**False (✗)
- **相关知识点：**体系结构设计决策 (Architectural Design Decisions)
非公式化 (No formulaic process): 没有公式化的体系结构设计过程。最好将体系结构设计看作是一系列需要做出的决策 (Decisions)，而不是一系列活动。
- **解析：**体系结构设计是一个创造性过程，依赖于系统类型和架构师经验，并不存在通过“公式化”步骤就能完成的流程。

5. 判断题

题目：【第6章】 The choice of architectural style and structure should depend on the functional requirements of the system because of the close relationship between them.

- **答案：**False (✗) (Depends more on non-functional requirements)
- **相关知识点：**体系结构设计决策 (Architectural Design Decisions)
非功能性需求 (Non-functional requirements): 体系结构风格的选择应取决于系统的非功能性需求 (如性能、安全性、可用性)，因为它们之间关系密切。
- **解析：**虽然功能需求很重要，但在选择架构风格 (如分层、C/S) 时，主要驱动力是性能、安全、可维护性等非功能性需求。

6. 多选题

题目：【第6章】 Which views belong to Krutchen's well-known 4+1 view model of software architecture, and are linked through the common use cases or scenarios?

- **答案：**A, B, C, D (全选)
- **相关知识点：**体系结构视图 (Architectural Views) - 4+1 视图模型
Krutchen 提出的 4+1 视图模型 包含四个基本视图: 逻辑视图 (Logical view)、进程视图 (Process view)、开发视图 (Development view)、物理视图 (Physical view)。它们通过用例或场景联系起来。
- **解析：**题目列出的四个选项正是 4+1 视图模型中的“4”个基本视图。

7. 单选题

题目：【第6章】 When you design a system which need to separate presentation and interaction from the system data, which architectural pattern is the most appropriate to be used?

- **答案：**A
- **相关知识点：**MVC 模式 (Model-View-Controller)
描述: 将演示 (Presentation) 和交互 (Interaction) 与系统数据 (System data) 分离。
- **解析：**MVC 模式的核心定义就是将显示/交互 (View/Controller) 与数据 (Model) 分离。

8. 单选题

题目：【第6章】 If the system to be designed should be organized so that each processing component carries out one type of data transformation, and the data flows from one component to another for processing, which architectural pattern should be used?

- **答案：**D
- **相关知识点：**管道和过滤器模式 (Pipe and Filter Pattern)
描述: 每个处理构件 (过滤器) 是离散的，执行一种类型的数据转换 (Data transformation)。数据从一个构件流向另一个构件 (管道)。
- **解析：**题目描述的“数据转换”和“数据流向另一个构件”正是管道-过滤器模式的典型特征。

9. 单选题

题目：【第6章】 When the system should be presented as a set of services, with each service delivered by a separate server, and the users access the servers to make use of them, which architectural pattern should be used?

- **答案：**B
- **相关知识点：**客户机-服务器模式 (Client-Server Pattern)
描述: 系统作为一组服务 (Services) 呈现，每个服务由单独的服务器 (Server) 提供。客户机 (Clients) 访问并使用这些服务。
- **解析：**关键词是“Services” (服务) 和“Server” (服务器)，以及用户访问这些服务，这对应 C/S 架构。

10. 填空题

题目：【第7章】 Software design and implementation is the stage in the software engineering process at which an

\$\$\$填空1\$\$\$

software system is developed.

- **答案：**executable / 可执行 / 可运行
- **相关知识点：**设计与实现基础
定义: 软件设计和实现是软件工程过程中开发可执行软件系统 (Executable software system) 的阶段。
- **解析：**设计和实现的最终目标是产出可以运行的代码，即“可执行的”系统。

11. 填空题

题目：For many types of application, it is now possible to buy off-the-shelf application systems that can be adapted and

\$\$\$填空1\$\$\$

to the users' requirements.

- **答案：**tailored / 剪裁
- **相关知识点：**设计与实现基础
构建或购买 (Build or buy): 对于许多类型的应用，现在可以购买现成的 (Off-the-shelf) 应用系统，并根据用户需求进行改编和剪裁 (Adapted and tailored)。
- **解析：**这是“购买 vs 构建”决策中的一个关键点，现成软件通常需要“tailored” (剪裁/定制) 以适应特定需求。

12. 填空题

题目：【第7章】 In object-oriented design, objects include both data and

\$\$\$填空1\$\$\$

to manipulate that data.

- **答案：**operations / operation / 操作
- **相关知识点：**面向对象设计 (Object-Oriented Design)
对象 (Objects): 包含数据 (Data) 和操作数据的操作 (Operations)。
- **解析：**对象的封装性体现在它同时包含状态 (数据) 和行为 (操作)。

13. 填空题

题目：【第7章】 Understanding the context of the software that is being designed lets you establish the

\$\$\$填空1\$\$\$

of the system, which helps you decide what features are imple

mented in the system and what features are in other associated systems.

- **答案:** boundaries / boundary / 边界
- **相关知识点:** 上下文与交互模型
上下文模型 (Context model): 理解上下文有助于建立系统的边界 (Boundaries), 从而决定哪些功能属于系统内, 哪些属于系统外。
- **解析:** 上下文分析的主要目的之一就是划定系统的“边界”。

14. 多选题

题目: 【第 7 章】 In the system context model shown below, which associations present 1:1 relationships?

- **答案:** B, C
- **相关知识点:**
系统上下文 (System Context)
PPT 中的图示 (Figure 7.1) 描述了气象站系统的环境。根据图示的基数 (Cardinality) 信息:
 - Control system (控制系统) 与 Weather information system (天气信息系统) 在环境中通常作为单一实体存在。
 - Weather information system 与 Satellite (卫星) 之间的通信通常也是单一链路对应的。
- **解析:** 虽然题目没有显示图片, 但根据标准答案 B (Control system---Weather information system) 和 C (Weather information system---Satellite), 这反映了在典型的气象站系统上下文模型中, 这些外部高层系统之间通常是 1 对 1 的交互关系 (例如, 一个控制中心对应一个信息处理中心, 一个信息中心对应一个卫星链路)。相比之下, 气象站本身通常是多个 (1..n)。

15. 单选题

题目: 【第 7 章】 Which option is the most appropriate result for identifying the object, attribute and operation from the statement: "Flight CA1842 arrived".

- **答案:** C
- **相关知识点:** 对象类识别 (Object Class Identification)
使用语法分析: 对象通常是名词 (实体), 属性是名词

- (特征), 操作是动词。
- **解析:**
 - "Flight" (航班) 是实体类 -> **Object**。
 - "CA1842" 是具体航班的标识号 -> **Attribute**。
 - "arrived" (到达) 是动作 -> **Operation**。

16. 单选题

题目: 【第 7 章】 Which statement is false about the sequence diagram shown below?

- **答案:** B
- **相关知识点:** 顺序模型 (Sequence models)
显示对象交互的顺序。对象在生命周期线上存在。
- **解析:**
 - 选项 B 声称 "When send(Report) occurs, the three objects... are all vanished" (当发送报告时, 三个对象都消失了)。
 - **错误原因:** 在顺序图中, 对象通常在交互过程中持续存在, 除非显式销毁。仅仅发送一个消息并不意味着对象“消失”或被销毁, 它们依然存在于内存中等待后续交互或直到生命周期结束。

17. 多选题

题目: In the state diagram shown below, from which states can one directly enter the "Running" state?

- **答案:** A, B, C, D (全选)
- **相关知识点:**
状态机模型 (State machine models)
PPT Figure 7.8 (Weather station state diagram) 展示了状态转换。
 - Shutdown -> restart() -> Running
 - Configuring -> configuration done -> Running
 - Collecting -> collection done -> Running
 - Transmitting -> transmission done -> Running
- **解析:** 根据气象站的状态图, 系统可以从关机(Shutdown)、配置(Configuring)、采集(Collecting)和传输(Transmitting)状态通过特定事件回到运行(Running)状态。

18. 单选题

题目: 【第 7 章】 Which statement is false about interface specification?

- **答案:** D
- **相关知识点:** 接口规范 (Interface Specification)
对象和接口之间不是简单的 1:1 关系 (There is not a simple 1:1 relationship between objects and interfaces)。
- **解析:** 选项 D 说 "There is a simple 1:1 relationship between objects and interfaces", 这与知识点直接冲突。一个对象可以实现多个接口, 一个接口也可以被多个对象实现。

19. 判断题

题目: 【第 7 章】 Design patterns are usually associated with object-oriented design.

- **答案:** True (✔)
- **相关知识点:** 设计模式 (Design Patterns)
设计模式通常与面向对象设计 (Object-oriented design) 相关联。发布的模式通常依赖于继承和多态等对象特性。
- **解析:** 虽然设计模式的思想可用于任何设计, 但在软件工程文献中, 它们通过 GoF 的书与 OOD 紧密绑定。

20. 多选题

题目: 【第 7 章】 Which options are the essential elements of design patterns defined by The Gang of Four?

- **答案:** A, B, C, D (全选)
- **相关知识点:**
设计模式 (Design Patterns)
四人帮 (Gang of Four) 定义四个基本要素:
 - A name (名称)
 - A description of the problem area (问题描述)
 - A solution description (解决方案描述)
 - A statement of the consequences (后果陈述)
- **解析:** 这是 GoF 设计模式定义的标准四要素。

21. 单选题

题目: 【第 7 章】 When it is required to separate the display of the state of an object from the object itself and allows alt

ernative displays to be provided, which design pattern should be used?

- **答案:** A
- **相关知识点:** 观察者模式 (Observer Pattern)
描述: 将对象状态的显示 (Display of the state) 与对象本身分离, 并允许提供可替代的显示 (Alternative displays)。
- **解析:** 题目描述完全符合观察者模式的定义, 用于解决数据源与多个视图之间的同步问题。

22. 填空题

题目: 【第 7 章】 Production software does not usually execute on the same computer as the software development environment. Rather, you develop it on one computer (the host system) and execute it on a separate computer (the

\$\$填空 1\$\$
system).

- **答案:** target / 目标
- **相关知识点:** 实现问题 (Implementation Issues)
宿主机-目标机开发 (Host-target development): 开发在宿主机 (Host system) 上进行, 执行在目标机 (Target system) 上进行。
- **解析:** 这是嵌入式系统或跨平台开发中的标准术语: Host (开发端) vs Target (运行端)。

第三部分: 重点案例分析 -
Mentcare 系统

本部分针对教材中的 Mentcare (Mental Health Care - Patient Management System) 案例进行详解。这通常是关于分层体系结构 (Layered Architecture) 和系统建模的重要考点。

(1) Mentcare 系统简介 (Introduction)

- **定义:** Mentcare (有时称为 MHC-PMS) 是一个**心理健康病人信息系统 (Mental Health Patient Information System)**。它是一个用于维护和管理因心理健康问题咨询医生的病人信息的医疗信息系统。
- **主要功能:**
 - **生成管理信息 (Generate management information):** 帮助医疗服务管理者按月监控系统表现。

- **医疗记录管理:** 存储和检索病人的咨询、诊断和治疗记录。
- **使用场景:** 这类系统通常用于诊所, 供临床医生 (Clinicians)、护士和行政人员使用。
- **关键特性:** 由于涉及敏感的医疗数据, 系统对**隐私 (Privacy)** 和**安全性 (Safety)** 有极高要求。

(2) Mentcare 的上下文模型 (Context Model)

- **知识点:** 上下文模型展示了目标系统与其环境 (外部系统) 的边界。
- **Mentcare 的环境:** Mentcare 不是孤立存在的, 它通常与以下外部系统交互:
 - **预约系统 (Appointments System):** 管理病人的预约时间。
 - **病人记录系统 (Patient Record System):** 医院总部的中央系统, 存储更广泛的病人病历 (Mentcare 可能只是一个专门的子系统或与其同步)。
 - **处方系统 (Prescription System):** 用于开具和管理处方药。
 - **卫生行政报告系统 (Health Authority Reporting System):** 用于上报统计数据。
- **图示描述:** 在上下文图 (Context Diagram) 中, Mentcare 位于中心, 四周通过简单的关联线连接上述外部系统。

(3) Mentcare 的交互模型 (Interaction Model)

- **知识点:** 交互模型展示系统运行时对象或子系统之间的交互 (通常使用 UML 序列图)。
- **典型场景:** “更新病人信息”或“查看病人数据”。
- **流程示例 (Transferring Data):**
 - 用户 (Receptionist) 通过**用户界面**输入认证信息。
 - 系统通过**安全管理器 (Security Manager)** 验证用户。
 - 用户请求更新某个病人的数据。
 - 系统从**数据库**中检索病人记录。
 - 用户修改数据并提交。
 - 系统将更新后的数据写回数据库, 并可能向**审计日志 (Audit Log)** 发送记录。

(4) Mentcare 的结构模型 (Structural Model)

- **核心考点:** Mentcare 是**分层体系结构 (Layered Architecture)** 的典型应用案例。
- **分层设计 (Web-based Layered Architecture):**

- **顶层 (Top Layer):** Web 浏览器 / 用户界面 (Web browser)。提供用户访问入口。
- **第二层 (Layer 2):** 登录/认证与用户界面管理 (Login/Authentication, UI Management)。处理安全验证和界面逻辑。
- **第三层 (Layer 3):** 业务逻辑 / 应用服务 (Application Logic/Services)。这是核心层, 包含:
 - 4. 病人管理 (Patient management)
 - 5. 预约管理 (Appointment management)
 - 6. 报告生成 (Report generation)
- **底层 (Bottom Layer):** **数据库系统 (Database System)**。存储病人数据、事务记录等。
- **为什么使用分层?** : 因为它允许每一层独立开发和维护 (例如, 只要数据库接口不变, 可以随时更换数据库而不影响上层业务逻辑)。

第 8 章 - 软件测试 (Software Testing) 重点复习资料

第一部分: 核心知识点整理 (Key Concepts)

1. 什么是软件测试? (What is Software Testing?)

- **定义 (Definition):**
测试旨在展示程序做了它打算做的事情, 并在投入使用前发现程序缺陷。
 - *Testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use.*
- **本质:** 当你测试软件时, 你是使用 **人造数据 (Artificial data)** 执行程序。
- **Dijkstra 的名言:**
“测试只能证明错误的存在, 而不能证明没有错误。”
 - *“Testing can only show the presence of errors, not their absence”.*

2. 程序测试的目标 (Program Testing Goals)

- **确认测试 (Validation testing):**
 - 向开发者和客户展示软件满足其需求。
 - *Demonstrate that the software meets its requirements*
 - 期望系统使用反映预期使用的测试用例正确执行。

- **缺陷测试 (Defect testing):**
 - 发现软件行为不正确、不理想或不符合规范的输入或输入序列。
 - *Find inputs where the behavior is incorrect to expose defects*
 - 旨在根除系统崩溃、数据损坏等不良行为。

3. 验证与确认 (Verification vs Validation)

- **区别 (Difference):**
 - **验证 (Verification):** “我们是否在正确地构建产品?” (“Are we building the product *right*”)
 - 检查软件是否满足其规定的功能和非功能需求。
 - **确认 (Validation):** “我们是否在构建正确的产品?” (“Are we building the *right* product”)
 - 确保软件满足 **客户的期望 (Customer's expectations)**。
- V & V 的目标: 建立软件系统“适合其目的”的 **信心 (Confidence)**。信心水平取决于:
 - **软件目的 (Software purpose):** 关键软件需要高可靠性。
 - **用户期望 (User expectations)**。
 - **市场环境 (Marketing environment)**。

4. 检查与测试 (Inspections vs Testing)

- **检查 (Inspections):**
 - 是一种 **静态 (Static)** V & V 技术, **不需要执行 (Don't need to execute)** 软件。
 - 可以检查系统需求、设计模型、源代码等任何 **可读表示 (Readable representation)**。
 - **优势:**
 - 因为不运行系统, 不用担心错误间的 **相互影响 (Interactions between errors)**。
 - 可以检查 **不完整的版本 (Incomplete versions)**。
 - 可以考虑更广泛的质量属性 (如合规性、可维护性)。
- **测试 (Testing):**
 - 需要执行程序。
 - **优势:** 擅长发现由于不同部分之间 **意外交互 (Unexpected interactions)**、**时序问题 (Timing problems)**

或 **性能问题 (System performance)** 引起的缺陷。检查不擅长发现这些。

5. 软件测试过程 (Software Testing Process)

6. **设计测试用例 (Design test cases)** -> 不可自动化 (需要理解系统意图)。
7. **准备测试数据 (Prepare test data)**。
8. **使用测试数据运行程序 (Run program with test data)** -> 可自动化。
9. **比较结果与测试用例 (Compare results to test cases)** -> 可自动化。

6. 测试的三个阶段 (Three Stages of Testing)

2. **开发测试 (Development testing):** 系统在开发期间进行测试, 以发现 bug 和缺陷。
3. **发布测试 (Release testing):** 一个独立的测试团队在系统发布给用户之前测试系统的完整版本。
4. **用户测试 (User testing):** 用户或潜在用户在自己的环境中测试系统。

7. 开发测试的三个阶段 (Stages of Development Testing)

- **单元测试 (Unit testing):**
 - 测试单个程序单元或对象类。
 - 重点在于测试对象或方法的 **功能 (Functionality of objects or methods)**。
 - 应设计测试以提供对对象所有特性的 **覆盖 (Coverage)**。
- **构件/组件测试 (Component testing):**
 - 集成多个单元以创建复合组件。
 - 重点在于测试提供访问组件功能的 **组件接口 (Component interfaces)**。
- **系统测试 (System testing):**
 - 集成部分或所有组件, 将系统作为一个整体进行测试。
 - 重点在于测试 **组件的交互 (Component interactions)**。

8. 测试策略 (Testing Strategies)

- **黑盒测试 (Black-box testing):** 基于系统规范识别等价划分, **不需要 (Don't need)** 了解系统如何工作 (代码)。
- **白盒测试 (White-box testing):** 查看程序 **代码 (Code)** 来

寻找其他可能的测试。

9. 等价划分 (Equivalence Partitioning)

- 将输入数据看作具有共同特征的集合。
- **选择规则:** 选择划分 **边界 (Boundaries)** 上的测试用例, 加上划分中点附近的用例。
- **例如:** 输入区间 [3, 9]。测试应包括边界 (3, 9), 越界值 (2, 10) 以及中间值。

10. 接口测试 (Interface Testing)

- **接口类型 (Interface types):**
 - **参数接口 (Parameter interfaces)**
 - **共享内存接口 (Shared memory interfaces)**
 - **过程接口 (Procedural interfaces)**
 - **消息传递接口 (Message passing interfaces)**
- **接口错误类型 (Interface errors):**
 - **接口误用 (Interface misuse)**
 - **接口误解 (Interface misunderstanding)**
 - **时序错误 (Timing errors)**

11. 测试驱动开发 (Test-driven development, TDD)

- **定义:** 一种交织 (Interleave) 测试和代码开发的程序开发方法。
- **流程:** 编写测试 -> 运行失败 -> 编写代码 -> 运行通过。
- **原则:** 在你开发的代码通过所有测试之前, 不要开始下一个增量的开发。
- **好处 (Benefits):**
 - **代码覆盖 (Code coverage)**。
 - **回归测试 (Regression testing):** 可以随时运行测试, 检查更改是否引入了新 bug。
 - **简化调试 (Simplified debugging)**。
 - **系统文档 (System documentation)**。

第二部分: 习题解析 (Quiz Analysis)

1. 填空题

Testing is intended to show that a program does what it is intended to do and to discover program [填空 1] before it is put into use.

- 答案:

defects / defect / 缺陷

- 涉及知识点:
第8章 - 什么是软件测试? (Definition of Software Testing)
 - 测试旨在展示程序做了它打算做的事情, 并在投入使用前发现程序缺陷。
 - *Testing is intended to show that a program does what it is intended to do and to discover program **defects** before it is put into use.*
- 解析:
这是软件测试定义的直接填空。测试的核心目的除了验证功能, 就是在发布前发现缺陷 (defects)。

2. 判断题

Testing can only show the presence of errors, not their absence.

- 答案:
True (✓)
- 涉及知识点:
第8章 - Dijkstra 的名言
 - Dijkstra 的名言: “测试只能证明错误的存在, 而不能证明没有错误。”
 - *"Testing can only show the **presence** of errors, not their **absence**".*
- 解析:
这是一个经典的软件工程原则。无论测试多少次, 你只能证明程序当前有错或者当前测试没报错, 但永远无法逻辑上证明程序里没有隐藏的错误。

3. 单选题

Which statement is false about the V & V (Verification and Validation) ?

- A. Software verification is the process of checking that the software meets its stated functional and non-functional requirements.
 - B. Verification and validation are not the same thing, although they are often confused.
 - C. The aim of software verification is to ensure that the software meets the customer's expectations.
 - D. Validation means "Are we building the right product?", while verification means "Are we building the product right".
- 答案:
C (Verification assures it meets requirements; Validation

assures it meets customer expectations)

- 涉及知识点:
第8章 - 验证与确认 (Verification vs Validation)
 - **验证 (Verification)**: 检查软件是否满足其规定的需求 (requirements)。“我们在正确地构建产品吗?”
 - **确认 (Validation)**: 确保软件满足客户的期望 (customer's expectations)。“我们在构建正确的产品吗?”
- 解析:
选项 C 是错误的, 因为它把“满足客户期望”归因为 Verification。实际上, “满足客户期望”是 Validation (确认) 的目标。Verification 关注的是符合规格说明书 (Requirements)。

4. 填空题

The goal of V&V (Verification and Validation) is to establish [填空 1] that the software system is "fit for purpose."

- 答案:
confidence / 信心
- 涉及知识点:
第8章 - V & V 的目标
 - V & V 的目标是建立软件系统“适合其目的”的 **信心 (Confidence)**。
 - *The goal of V&V is to establish **confidence** that the software system is "fit for purpose."*
- 解析:
V&V 不能保证零缺陷, 只能建立足够的信心, 让用户和开发者相信系统是可用的。

5. 判断题

Testing mostly focus on the source code of a system, but any readable representation of the software, such as its requirements or a design model, can be tested, while Inspections can only be applied to executable programs and system prototypes.

- 答案:
False (✗)
- 涉及知识点:
第8章 - 检查与测试 (Inspections vs Testing)
 - **检查 (Inspections)**: 是一种静态技术, 可以应用于任何 **可读表示 (Readable representation)** (如需求、设计模型、源码), **不需要执行** 软件。
 - **测试 (Testing)**: 需要执行程序, 主要针对可执行代码。

- 解析:
题目把 Testing 和 Inspections 的适用范围说反了。题目说“Testing 可以测文档, Inspection 只能测可执行程序”, 这完全错误。Inspection 才是用来检查文档和代码 (作为文本) 的, Testing 必须运行代码。

6. 单选题

Which statement is false about inspections/reviews and testing ?

- A. Compared to testing, inspections/reviews are "static" V & V techniques in which you don't need to execute the software to verify it.
- B. Incomplete versions of a system can be inspected without additional costs, while you need to develop specialized test harnesses to test the parts of the program.
- C. For inspection, you don't have to worry about interactions between errors, a single inspection session can discover many errors in a system.
- D. Testing are not good for discovering defects that arise because of unexpected interactions between different parts of a program, timing problems, or problems with system performance.

- 答案:
D (Testing is good for finding interaction/timing defects; Inspections are bad at it)
- 涉及知识点:
第8章 - 检查与测试的对比优势
 - **检查 (Inspections)**: 不擅长发现动态问题。
 - **测试 (Testing)**: 擅长发现由于不同部分之间 **意外交互 (Unexpected interactions)**、**时序问题 (Timing problems)** 或 **性能问题 (System performance)** 引起的缺陷。
- 解析:
选项 D 说“测试不擅长发现交互和时序问题”, 这与知识点相反。测试是动态运行的, 正是为了发现这些运行时才会出现的交互和性能问题。检查 (静态看代码) 才是不擅长发现这些的。

7. 单选题

In the model of the software testing process shown below, which activity can not be automated ? (Design test cases -> Prepare test data -> Run program -> Compare results)

- A. Design test cases
- B. Prepare test data
- C. Run program with test data

D. Compare results to test cases

- 答案:
A
- 涉及知识点:
第8章 - 软件测试过程 (Software Testing Process)
 - **设计测试用例 (Design test cases)**: 不能自动化。需要人去理解系统意图和需求。
 - **运行程序 (Run program) 和 比较结果 (Compare results)**: 通常可以自动化。
- 解析:
机器无法理解“系统应该做什么”或者“业务逻辑是什么”，所以设计测试用例必须由人来完成。一旦设计好，后续的运行和比对可以交给脚本自动执行。

8. 填空题

The three test stages are: development testing, [填空 1] testing, and user testing.

- 答案:
release / Release / 发布
- 涉及知识点:
第8章 - 测试的三个阶段 (Three Stages of Testing)
 - **开发测试 (Development testing)**
 - **发布测试 (Release testing)**: 在发布给用户前，由独立团队测试完整版本。
 - **用户测试 (User testing)**
- 解析:
这是测试生命周期的三个主要阶段，位于开发和用户使用中间的环节就是发布测试。

9. 填空题

The three stages of development testing are: unit testing, [填空 1] testing, and system testing.

- 答案:
Component / component / 构件 / 组件
- 涉及知识点:
第8章 - 开发测试的三个阶段
 - **单元测试 (Unit testing)**
 - **构件/组件测试 (Component testing)**: 集成多个单元，测试组件接口。
 - **系统测试 (System testing)**
- 解析:

开发测试从最小的单元开始，然后是组件（由单元组成），最后是整个系统。

10. 填空题

When testing object classes, you should design tests to provide [填空 1] of all features of the object.

- 答案:
coverage / 覆盖
- 涉及知识点:
第8章 - 单元测试 (Unit Testing)
 - 测试对象类时，应设计测试以提供对对象所有特性的 **覆盖 (Coverage)**。这包括测试所有操作、属性设置和状态。
- 解析:
单元测试的一个关键指标就是“覆盖率”，确保对象的所有功能点都被测到了。

11. 填空题

For the three stages of development testing, unit testing should focus on testing the functionality of objects or methods, component testing should focus on testing the component [填空 1] that provide access to the component functions, and system testing should focus on testing component interactions.

- 答案:
interfaces / interface / 接口
- 涉及知识点:
第8章 - 开发测试各阶段的重点
 - 单元测试: 功能 (Functionality)。
 - **构件测试 (Component testing)**: 重点在于测试提供访问组件功能的 **组件接口 (Component interfaces)**。
 - 系统测试: 交互 (Interactions)。
- 解析:
组件通常通过接口对外提供服务，因此组件测试的核心就是验证接口行为是否符合规范。

12. 多选题

For the state diagram and object interface shown below (WeatherStation), if the sequence of state transitions is: Shutdown->Configuring->Running->Summarizing, then which methods (or functions) can be tested?

A. reconfigure()

B. shutdown()

C. reportWeather()

D. powerSave()

- 答案:
A, C, D (reconfigure/powerSave in Shutdown, reportWeather in Running. 'shutdown' is the transition into Shutdown, usually tested when entering)
- 涉及知识点:
第8章 - 状态模型测试 (Testing with State Models)
 - 在测试中，我们需要触发特定的状态转换。
 - 根据 PPT 中的状态图示例（虽然此处未直接显示图，但根据逻辑推断）：
 - Shutdown 状态通常允许配置 (reconfigure) 或省电设置 (powerSave)。
 - Running 状态允许报告天气 (reportWeather)。
- 解析:
 - 在 Shutdown -> Configuring 过程中，会测试配置相关方法 (A)。
 - 在 Running 状态下，气象站工作，会测试 reportWeather (C)。
 - powerSave 通常也是在非全速运行或特定状态下测试的 (D)。
 - shutdown() 是进入关机状态的方法，题目给出的序列是从 Shutdown 开始，或者是在 Running 之后没有明确提到再次调用 shutdown 结束，重点在于序列中间经历的状态所支持的方法。

13. 单选题

If a program has an input that is within the interval [3, 9], then what is the most appropriate set of numbers used for testing?

A. 2, 3, 4, 6, 8, 9, 10

B. 4, 5, 6, 7, 8, 9

C. 3, 6, 9

D. 3, 5, 6, 7, 9

- 答案:
A (Includes boundaries 3, 9 and out-of-bounds 2, 10)
- 涉及知识点:
第8章 - 等价划分 (Equivalence Partitioning)
 - **测试用例选择规则**: 选择划分 **边界 (Boundaries)** 上的测试用例，加上划分中点附近的用例。同时也需要测试 **无效划分 (Invalid partitions)** 的值。
- 解析:

- 有效区间: [3, 9]。边界是 3 和 9。
- 无效区间 (越界): <3 (如 2) 和 >9 (如 10)。
- 选项 A 包含了:
 - 3. 边界值: 3, 9
 - 4. 刚越界的值: 2, 10
 - 5. 中间值: 4, 6, 8
- 这是最完整的边界值分析测试集。其他选项缺少越界值或边界值。

14. 判断题

For black-box testing, you don't need any knowledge of how the system works. While for white-box testing, you may look at the code of the program to find other possible tests.

- 答案:
True (✓)
- 涉及知识点:
第 8 章 - 黑盒与白盒测试 (Black-box vs White-box)
 - **黑盒测试**: 基于规范, **不需要 (Don't need)** 了解系统内部工作原理。
 - **白盒测试**: 查看程序 **代码 (Code)** 来寻找测试用例。
- 解析:
题目准确描述了两种测试策略的核心区别: 是否看代码/内部逻辑。

15. 填空题

Interface types include: parameter interfaces, shared memory interfaces, [填空 1] interfaces, and message passing interfaces.

- 答案:
Procedural / procedural / 过程
- 涉及知识点:
第 8 章 - 接口类型 (Interface Types)
包含四种:
 - 参数接口 (Parameter)
 - 共享内存接口 (Shared memory)
 - **过程接口 (Procedural interfaces)**
 - 消息传递接口 (Message passing)
- 解析:
考察对四种标准接口类型的记忆。过程接口是指一个组件封装了一组过程供其他组件调用。

16. 填空题

There are three types of interface errors: interface misuse, [填空 1], and timing errors.

- 答案:
interface misunderstanding / Interface misunderstanding / 接口误解
- 涉及知识点:
第 8 章 - 接口错误类型 (Interface Errors)
 - 接口误用 (Interface misuse)
 - **接口误解 (Interface misunderstanding)**: 调用组件对被调用组件的行为做出了错误的假设。
 - 时序错误 (Timing errors)
- 解析:
考察三种接口错误的分类。误用是参数传错了, 误解是理解错了功能, 时序是数据更新不及时。

17. 填空题

Exhaustive testing is impossible, so testing has to be based on a [填空 1] of possible test cases.

- 答案:
subset / 子集
- 涉及知识点:
第 8 章 - 穷尽测试 (Exhaustive Testing)
 - 穷尽测试 (测试所有可能的输入组合) 是不可能的。
 - 测试必须基于可能测试用例的 **子集 (Subset)**。
- 解析:
因为输入空间通常是无限的, 所以只能选择一个代表性的“子集”进行测试。

18. 单选题

Which statement is false about Test-driven development(TDD) ?
A. TDD is an approach to program development in which you interleave testing and code development.
B. You develop the code incrementally, along with a set of tests for that increment.
C. You may start working on the next increment even when the code that you have developed haven't passed all of its tests.
D. TDD has now gained mainstream acceptance and may be used in both agile and plan-based processes.

- 答案:
C (Must pass tests before moving on)
- 涉及知识点:
第 8 章 - 测试驱动开发 (TDD) 原则
 - **原则**: 在你开发的代码通过所有测试之前, **不要 (Don't)** 开始下一个增量的开发。
 - *You don't start working on the next increment until the code passes all of its tests.*
- 解析:
C 选项说“即使代码没通过测试也可以开始下一个增量”, 这完全违反了 TDD 的核心原则。TDD 要求必须“红 (失败) -> 绿 (通过)”之后才能继续。

19. 填空题

A test suite is developed incrementally as a program is developed. You can always run [填空 1] tests to check that changes to the program have not introduced new bugs.

- 答案:
regression / 回归
- 涉及知识点:
第 8 章 - TDD 的好处 / 回归测试
 - **回归测试 (Regression testing)**: 因为测试套件是增量开发的, 你可以随时运行回归测试, 检查程序更改是否引入了新 bug。
- 解析:
“Regression testing”指的是重新运行以前的测试, 以确保新代码没有破坏旧功能。这是 TDD 的主要优势之一。

软件工程复习资料 (第 9, 10, 15, 22 章)

第一部分: 各章节重点知识点整理 (Key Knowledge Points)

【第 9 章】软件演化 (Software Evolution)

1. 演化与维护的概念 (Evolution vs. Servicing)

- **软件演化 (Software Evolution)**: 指对软件架构和功能进行 **重大改变 (Significant changes)** 的阶段。在这个阶段, 软件会发布新版本。
- **软件维修 (Software Servicing)**: 指软件退役前的阶段, 此

时只进行相对**较小但必要** (Small but essential) 的修改 (如修补 Bug)，不再进行重大的功能升级。

2. 演化的重要性与成本 (Importance and Cost)

- **棕地系统 (Brownfield Systems)**: 在企业系统中，当单个软件系统是更广泛的“**系统之系统 (System of Systems)**”的一部分时，软件演化特别昂贵。修改一个系统可能导致环境中的其他系统也需要随之演化。
- **演化驱动力: 变更建议 (Change proposals)** 是系统演化的驱动力。

3. 遗留系统 (Legacy Systems)

- **定义**: 指依赖于不再用于新系统开发的语言和技术的**旧系统 (Older systems)**。它们通常维护了很长时间，结构已经退化。
- **组成要素 (Elements)**:
 - 系统硬件 (System hardware)
 - 支持软件 (Support software): 如操作系统、工具。
 - 应用软件 (Application software)
 - 应用数据 (Application data): 可能不一致、重复或分散。
 - 业务过程 (Business processes)
 - 业务策略和规则 (Business policies and rules)
- **分层模型 (Layers)**: 从底层的硬件到顶层的业务过程，每一层都依赖于下一层。
- **替换风险 (Replacement Risks)**: 替换遗留系统通常**昂贵且充满风险 (Expensive and risky)**.
 - 原因: 很少有完整的系统规格说明 (Complete specification); 业务规则嵌入在代码中; 新开发本身有风险。
- **变更成本高昂的原因**: 代码风格不一致、编程语言过时、文档缺失或过时、系统结构退化、数据结构不兼容。

4. 软件维护 (Software Maintenance)

5. **定义**: 系统交付后对其进行变更的通过程。

6. 三种类型 (Types of Maintenance):

- **故障修复 (Fault repairs)**: 修复 Bug 和漏洞 (约占 24%)。
- **环境适应 (Environmental adaptation)**: 适应新的平台或环境 (约占 19%)。
- **功能增加 (Functionality addition)**: 添加新功能以支持新需求 (约占 58%, **成本最高**)。

7. **维护成本规律**: 在维护阶段添加新功能通常比在初始开发阶段实现相同功能更昂贵。

【第 10 章】可依赖系统 (Dependable Systems)

1. 可依赖性及其重要性 (Dependability)

- **定义**: 涵盖可用性、可靠性、安全性 (Safety) 和信息安全性 (Security) 等相关系统属性的术语。
- **重要性**: 系统的可依赖性通常比其**详细功能 (Detailed functionality)** 更重要。因为系统失效会影响大量用户、导致用户拒绝使用、带来巨大的经济损失或信息丢失。

2. 可依赖性的 5 个主要属性 (Principal Dimensions)

- **可用性 (Availability)**: 系统在任何给定时间都能运行并提供服务的概率。
- **可靠性 (Reliability)**: 系统在给定时间内按预期正确提供服务的概率。
- **安全性 (Safety)**: 系统不造成人员伤亡或环境破坏的可能性 (判断)。
- **信息安全性 (Security)**: 系统抵御意外或蓄意入侵的能力 (判断)。
- **韧性 (Resilience)**: 系统在遭遇破坏性事件时维持关键服务连续性的能力。

3. 成本与可依赖性曲线 (Cost/Dependability Curve)

- **规律**: 随着可依赖性要求的提高，成本呈指数级增长。
- **低可依赖性阶段**: 如果软件不是很可靠，通过更好的工程方法可以以**较低成本获得显著提升**。
- **高可依赖性阶段**: 如果已经采用了良好的实践，进一步提升的成本将非常高，而从中获得的收益会减少。

4. 社会技术系统 (Sociotechnical Systems)

- 软件系统不是孤立的，而是更广泛的社会技术系统的一部分。
- 包括: 硬件、软件、人员 (People)、过程 (Processes)、组织 (Organizations) 等。

5. 冗余与多样性 (Redundancy and Diversity)

- **冗余 (Redundancy)**: 包含备用容量 (如备用服务器)，当部分系统失效时可以使用。
- **多样性 (Diversity)**: 冗余组件使用不同类型 (如不同操作系统、不同算法)，以减少因相同原因同时失效的概率。

【第 15 章】软件复用 (Software Reuse)

1. 复用的层级 (Levels of Reuse)

- **系统复用 (System reuse)**: 复用完整的系统 (可能作为“系统之系统”的一部分)。
- **应用复用 (Application reuse)**: 复用特定的应用，可能需要配置。
- **构件复用 (Component reuse)**: 复用应用的一个子系统或构件。
- **对象和函数复用 (Object and function reuse)**: 复用实现单一功能的类或函数 (如数学库)。

2. 复用的优缺点

- **优点 (Benefits)**:
 - 加速开发 (Accelerated development)
 - 有效利用专家知识 (Effective use of specialists)
 - 提高可依赖性 (Increased dependability): 复用的代码经过了测试。
 - 降低开发成本 (Lower development costs)
 - 符合标准 (Standards compliance)
- **问题 (Problems)**:
 - 创建和维护构件库成本高。
 - 查找、理解和适配构件困难。
 - “**非在此发明**”综合症 (Not-invented-here syndrome): 工程师倾向于重写代码，因为他们认为自己能写得更好，或者不信任他人的代码。

3. 软件产品线 (Software Product Lines)

- 5. **定义**: 一组具有**公共架构 (Common architecture)** 和**共享构件 (Shared components)** 的应用程序，每个应用都针对特定的客户需求进行了专门化。
- 6. **核心**: 核心系统 (Core system) 可被配置和适配。

【第 22 章】项目管理 (Project Management)

1. 管理的必要性

- 职业软件工程总是受到**组织预算 (Organizational budget)** 和**进度约束 (Schedule constraints)** 的限制，因此需要项目管理。

2. 项目管理成功的标准 (Success Criteria)

- 在约定时间交付软件 (Deliver at the agreed time)。

- 将总成本控制在预算内 (Keep costs within budget)。
- 交付满足客户期望的软件 (Meet customer's expectations)。
- 维持一个有凝聚力且运作良好的开发团队 (Maintain a coherent and well-functioning team)。

3. 团队协作 (Teamwork)

- **团队凝聚力 (Group cohesiveness):** 成功的团队不仅仅是技能的集合，还是一个有凝聚力的整体。成员忠于团队，将团队利益置于个人之上。

第二部分：第四次作业题目解析 (Quiz Analysis)

试卷信息：共 15 题，满分 25 分

1. 填空题 【第 9 章】

题目：Software evolution is particularly expensive in enterprise systems when individual software systems are part of a broader "[填空 1]"。

- **答案：**system of systems / system of systems. / brownfield systems
- **解析：**
 - **知识点：**演化的重要性 (Importance of evolution)。
 - **依据：**PPT 第 9 章指出，在企业系统中，当单个软件系统是更广泛的“系统之系统 (system of systems)”的一部分时（也称为棕地系统 brownfield system），软件演化特别昂贵。
 - **原因：**因为你不能仅仅考虑对一个系统的修改，还需要检查这些修改如何影响更广泛的系统环境。

2. 判断题 【第 9 章】

题目：In fact, the need for evolution may become obvious even before the system is deployed, so later releases may start development before the current version has even been released.

- **答案：**True (✓)
- **解析：**
 - **知识点：**螺旋模型 (Spiral model)。
 - **依据：**PPT 第 9 章提到，演化的需求甚至在系统部署之前就已经变得明显，因此后续版本 (later releases) 可能会在当前版本发布之前就开始开发。

3. 填空题 【第 9 章】

题目：Evolution is the phase in which significant changes to the software architecture and functionality are made. On the other hand, During [填空 1], the only changes that are made are relatively small but essential changes.

- **答案：**servicing
- **解析：**
 - **知识点：**演化与维修 (Evolution and servicing)。
 - **依据：**PPT 第 9 章区分了演化和维修。Evolution 涉及架构和功能的重大改变；而 Servicing (维修) 阶段所做的唯一改变是相对较小但必要的改变（如修补 Bug），此时系统正在走向退役。

4. 填空题 【第 9 章】

题目：Formal or informal system change [填空 1] are the driver for system evolution.

- **答案：**proposals / proposal
- **解析：**
 - **知识点：**演化过程 (Evolution processes)。
 - **依据：**PPT 第 9 章明确指出：“Formal or informal system change **proposals** are the driver for system evolution.”（正式或非正式的系统变更**建议**是系统演化的驱动力）。

5. 填空题 【第 9 章】

题目：[填空 1] systems are older systems that rely on languages and technology that are no longer used for new systems development.

- **答案：**Legacy
- **解析：**
 - **知识点：**遗留系统 (Legacy systems) 定义。
 - **依据：**PPT 第 9 章定义：Legacy systems are older systems that rely on languages and technology that are no longer used for new systems development.

6. 多选题 【第 9 章】

题目：Which options are the elements of a legacy system?

- A. System hardware
- B. Support software

- C. Application software
- D. Application data
- **答案：**A, B, C, D (全选)
- **解析：**
 - **知识点：**遗留系统的组成要素 (Elements of a legacy system)。
 - **依据：**PPT 第 9 章图表显示，遗留系统不仅仅是软件，还包括：System hardware (硬件), Support software (支持软件), Application software (应用软件), Application data (应用数据), Business processes (业务过程), Business policies and rules (业务策略)。选项 A、B、C、D 均在其中。

7. 单选题 【第 9 章】

题目：Which statement is false about the legacy systems?

- A. It is expensive and risky to replace legacy systems with more modern equivalent.
- B. An alternative way of looking at the components of a legacy system is as a series of layers.
- C. There is usually a complete specification of the legacy system.
- D. Legacy systems are particularly expensive to change.
- **答案：**C
- **解析：**
 - **知识点：**遗留系统替换风险 (Legacy system replacement)。
 - **依据：**PPT 第 9 章指出替换遗留系统昂贵且有风险的原因之一是：“There is **rarely** a complete specification of the legacy system.”（**很少**有完整的遗留系统规格说明）。
 - **原因：**选项 C 说“usually a complete specification”（通常有完整的规格说明），这与事实相反，因此是错误的陈述。

8. 填空题 【第 9 章】

题目：There are three types of software maintenance: fault repairs, [填空 1], and functionality addition.

- **答案：**environmental adaptation / Environmental adaptation
- **解析：**
 - **知识点：**维护类型 (Types of maintenance)。

- 依据: PPT 第 9 章列出了三种维护类型:
 - Fault repairs (故障修复)
 - Environmental adaptation (环境适应)
 - Functionality addition (功能增加)

9. 判断题 【第 10 章】

题目: The dependability of systems is usually more important than their detailed functionality.

- 答案: True (✓)
- 解析:
 - 知识点: 系统可依赖性 (System dependability)。
 - 依据: PPT 第 10 章明确指出: "The dependability of systems is usually more important than their detailed functionality." (系统的可依赖性通常比其详细功能更重要)。因为系统不可靠可能导致用户拒绝使用整个系统。

10. 多选题 【第 10 章】

题目: Which options belong to the dependability attributes?

- A. Availability
- B. Reliability
- C. Safety
- D. Security
- 答案: A, B, C, D (全选)
- 解析:
 - 知识点: 可依赖性属性 (Dependability properties)。
 - 依据: PPT 第 10 章图表显示, Dependability 主要包含五个属性: Availability (可用性), Reliability (可靠性), Safety (安全性), Security (信息安全性), Resilience (韧性)。选项 A、B、C、D 均属于此。

11. 判断题 【第 10 章】

题目: The cost/dependability curve below shows that, if your software is not very dependable, then even if the costs of improvement are much greater, the benefits from that improvement are less.

- 答案: False (✗)
- 解析:
 - 知识点: 成本/可依赖性曲线 (Cost/dependability curve)。

- 依据: PPT 第 10 章指出: 如果软件不是很可靠 (not very dependable), 你可以以相当低廉的成本获得显著的提升 (get significant improvements fairly cheaply)。
- 原因: 题目描述的情况是“高可依赖性阶段”的特征 (成本高收益低), 而不是“低可依赖性阶段”的特征。低可依赖性阶段应该是“成本低收益高”。因此题目描述错误。

12. 多选题 【第 15 章】

题目: Software units that are reused may be of radically different sizes, these include:

- A. System reuse
- B. Application reuse
- C. Component reuse
- D. Object and function reuse
- 答案: A, B, C, D (全选)
- 解析:
 - 知识点: 复用单元的大小 (Software units size)。
 - 依据: PPT 第 15 章列出了复用的不同层级/大小: System reuse (系统复用), Application reuse (应用复用), Component reuse (构件复用), Object and function reuse (对象和函数复用)。

13. 填空题 【第 15 章】

题目: "Some software engineers prefer to rewrite component s because they believe they can improve on them... The phenomenon described above is called '[填空 1]' syndrome."

- 答案: Not-invented-here / not-invented-here / Not invented here / not invented here / NIH / 不是在此发明
- 解析:
 - 知识点: 复用的问题 (Problems with reuse)。
 - 依据: PPT 第 15 章描述了 "Not-invented-here" syndrome (非在此发明综合症): 一些软件工程师更喜欢重写构件, 因为他们相信自己能改进它们, 这部分是出于信任问题, 部分是因为认为写原创软件更具挑战性。

14. 填空题 【第 22 章】

题目: Projects need to be managed because professional software engineering is always subject to organizational [填空 1] a

nd schedule [填空 2].

- 答案: budget (预算), constraints (约束/制约/constraint)
- 解析:
 - 知识点: 项目管理的必要性 (Why manage projects)。
 - 依据: PPT 第 22 章指出: Projects need to be managed because professional software engineering is always subject to organizational budget and schedule constraints. (项目需要管理, 因为职业软件工程总是受到组织预算和进度约束的限制)。

15. 多选题 【第 22 章】

题目: Which options belong to the success criteria for project management?

- A. Deliver the software to the customer at the agreed time.
- B. Keep overall costs within budget.
- C. Deliver software that meets the customer's expectations.
- D. Maintain a coherent and well-functioning development team.
- 答案: A, B, C, D (全选)
- 解析:
 - 知识点: 项目管理成功的标准 (Success criteria for project management)。
 - 依据: PPT 第 22 章列出了四条标准:
 - Deliver... at the agreed time (按时交付).
 - Keep overall costs within budget (预算内).
 - Deliver software that meets the customer's expectations (满足期望).
 - Maintain a coherent and well-functioning development team (维持良好团队).选项 A、B、C、D 完全对应。

软件工程简答题复习资料 (详细版)

1. 软件原型的概念和益处 (Concept and benefits of software prototyping)

来源章节: 第 2 章 - 软件过程 (Chapter 2 - Software Processes)

概念 (Concept)

软件原型 (Software Prototype) 是系统的一个初始版本 (Initial version)，它并不是最终产品，而是用于在开发早期阶段实现以下目的的工具：

- 演示概念 (Demonstrate concepts): 向用户或客户展示系统的基本构想。
- 尝试设计选项 (Try out design options): 测试不同的设计方案是否可行。
- 通常用于需求工程 (Requirements Engineering) 阶段以帮助引出和验证需求，以及在系统设计 (System Design) 阶段探索解决方案。

益处 (Benefits)

使用原型的核心优势在于它可以预见变更 (Anticipate Changes)，从而减少后续开发中的返工成本。具体益处如下：

- 改善系统可用性与用户参与：
 - 原型允许潜在用户直观地看到系统将如何支持他们的工作。通过实际操作原型，用户能更清楚地表达他们的需求，而不是面对抽象的文档。
- 明确与完善需求 (Clarify Requirements):
 - 发现新想法：用户在使用原型的過程中，往往会产生新的需求想法。
 - 识别优缺点：用户可以快速发现软件功能的强项和弱项。
 - 揭示错误：随着原型的开发和演示，可以有效地发现系统需求文档中的错误 (Errors) 和遗漏 (Omissions)，这些在纯文本审查中很难被发现。
- 降低设计风险 (Risk Reduction):
 - 在系统设计阶段，原型可以作为一种“设计实验”，用来检查提议的设计方案的可行性 (Feasibility)。例如，测试某个技术架构是否能满足性能要求。
- 更有效地应对变更：
 - 通过原型在早期发现问题，可以避免在系统开发后期（如实现或测试阶段）才发现需求错误，从而大大降低修复错误的成本 (Cost of rework)。

2. 敏捷宣言的内容 (Content of Agile Manifesto)

来源章节： 第 3 章 - 敏捷软件开发 (Chapter 3 - Agile Software Development)

敏捷方法的出现是为了应对快速变化的商业环境，减少传统计划

驱动方法的开销。敏捷开发的领导者们提出了敏捷宣言 (Agile Manifesto)，其核心包含四条价值观对比。

(答题时，请列出以下四条对比，并简要解释其含义)

- 个体和交互 胜过 过程和工具
 - (Individuals and interactions over processes and tools)
 - 解释：软件开发本质上是人的活动。虽然过程和工具很重要，但团队成员之间的沟通、协作以及技能才是项目成功的关键。过分依赖僵化的流程可能会阻碍团队的灵活性。
- 可工作的软件 胜过 详尽的文档
 - (Working software over comprehensive documentation)
 - 解释：传统方法往往要求在写代码前编写大量文档，但文档容易过时且无法给客户带来直接价值。敏捷方法强调尽早交付可以运行的软件，让客户看到实际进展，而不是仅仅得到一堆纸面设计。
- 客户协作 胜过 合同谈判
 - (Customer collaboration over contract negotiation)
 - 解释：与其在项目开始时通过合同严格限定需求（这样会导致对立关系），不如让客户直接参与到开发过程中（如作为团队一部分）。持续的协作能确保开发的软件真正满足客户不断变化的需求。
- 响应变化 胜过 遵循计划
 - (Responding to change over following a plan)
 - 解释：在不确定的商业环境中，严格遵循一个不再适用的计划是由于没有价值的。敏捷方法主张拥抱变化，即便是在开发后期，也应该欢迎需求变更，以提高客户的竞争优势。

注：敏捷宣言的结语是：“虽然右项也有价值，但我们认为左项更有价值。”

3. 验证与确认的概念及其区别 (Concepts of Verification and Validation and their difference)

来源章节： 第 8 章 - 软件测试 (Chapter 8 - Software Testing)

概念 (Concepts)

这两个概念通常统称为 V & V，是确保软件质量的两个互补过程：

- 验证 (Verification):

- 核心问题：“我们是否在正确地构建产品？” (“Are we building the product right?”)
- 定义：验证是检查软件是否满足其规定的功能和非功能需求 (Functional and non-functional requirements) 的过程。它关注的是软件开发过程中的每一步是否符合上一阶段的规格说明 (Specification)。
- 确认 (Validation):
 - 核心问题：“我们是否在构建正确的产品？” (“Are we building the right product?”)
 - 定义：确认是确保软件满足客户期望 (Customer's expectations) 的过程。它关注的是最终产品是否真正解决了用户的问题，是否符合用户的实际使用场景。

区别 (Difference)

- 关注点不同：
 - 验证关注的是一致性 (Conformance): 软件是否符合系统规格说明书 (Specification) 的定义。
 - 确认关注的是适用性 (Fitness for purpose): 软件是否真正满足了用户 (User) 的实际需求和意愿。
- 必要性来源：
 - 验证是必要的，但还不够。因为需求规格说明本身可能就是错的或者不完整的，它可能无法完全反映客户的真实愿望。因此，即使通过了所有的验证（完全符合规格说明），软件仍可能无法通过确认（用户觉得不好用或不是他们想要的）。

4. “四人帮”定义的的设计模式四个基本要素 (Four essential elements of design patterns defined by “The Gang of Four”)

来源章节： 第 7 章 - 设计和实现 (Chapter 7 - Design and Implementation)

根据“四人帮” (Gang of Four, GoF) 的经典著作，每一个设计模式都是对一个特定问题的解决方案的描述，以便在不同场景下复用。一个设计模式必须包含以下四个基本要素：

- 名称 (Name):
 - 这是对模式的一个有意义的引用 (A meaningful reference)。
 - 作用：它提供了一个通用的词汇表，让设计者在交流时能够用一个词概括复杂的解决方案（例如说“我

们用观察者模式吧”)，提高了沟通效率。

- **问题描述 (Problem Description):**
 - 描述了该模式适用的**问题领域 (Problem area)**。
 - **作用:** 它解释了**何时 (When)** 可以应用该模式。通常包括特定的设计情境、问题列表以及使用该模式必须满足的条件。
- **解决方案描述 (Solution Description):**
 - 描述了设计的组成部分，包括：
 - **部分 (Parts):** 涉及的类和对象。
 - **关系 (Relationships):** 它们之间如何关联。
 - **职责 (Responsibilities):** 各自负责什么。
 - **重要说明:** 这不是一个具体的代码实现，而是一个可以以不同方式实例化的**模板 (Template)**。它提供的是解决问题的抽象结构。
- **后果 (Consequences):**
 - 描述了应用该模式后的结果和**权衡 (Trade-offs)**。
 - **作用:** 使用模式通常会有利有弊 (例如增加了灵活性但可能增加了复杂性或性能开销)。这一部分帮助设计者理解在特定情况下使用该模式的影响，以决定是否值得使用。

附录：重点原文中英对照 (Original Text & Translation)

以下是课件中关于这四个知识点的**英文原话**，背诵这些句子可以在考试中作为关键定义使用。

1. Software Prototyping (Chapter 2)

- **Definition:** "A prototype is an **initial version of a system** used to **demonstrate concepts** and **try out design options**."
 - (定义：原型是系统的初始版本，用于演示概念和尝试设计选项。)
- **Benefits:**
 - "Prototypes allow potential users to **see how well the system supports their work**."
 - (原型允许潜在用户看到系统如何支持他们的工作。)
 - "They may **get new ideas for requirements** and find strength and weakness in the software."
 1. (他们可能会获得关于需求的新想法，并发现软件的优缺点。)
 - "As the prototype is developed, it may **reveal errors**

and omissions in the system requirements."

1. (随着原型的开发，可能会揭示系统需求中的错误和遗漏。)

2. Agile Manifesto (Chapter 3)

- "We have come to value:"
 - "Individuals and interactions over processes and tools"
 - (个体和交互 胜过 过程和工具)
 - "Working software over comprehensive documentation"
 - (可工作的软件 胜过 详尽的文档)
 - "Customer collaboration over contract negotiation"
 - (客户协作 胜过 合同谈判)
 - "Responding to change over following a plan"
 - (响应变化 胜过 遵循计划)
- "That is, while there is value in the items on the right, **we value the items on the left more**."
 - (也就是说，虽然右项也有价值，但我们更看重左项。)

3. Verification & Validation (Chapter 8)

- **Key Question:**
 - **Validation:** "Are we building the **right product**?"
 - (确认：我们在构建正确的产品吗？)
 - **Verification:** "Are we building the **product right**?"
 - (验证：我们在正确地构建产品吗？)
- **Definition:**
 - "Software verification is the process of checking that the software meets its **stated functional and non-functional requirements**."
 - (软件验证是检查软件是否满足其规定的功能和非功能需求的过程。)
 - "The aim of **software validation** is to ensure that the software meets the **customer's expectations**."
 - (软件确认的目标是确保软件满足客户的期望。)

4. Elements of Design Patterns (Chapter 7)

- "The Gang of Four defined the four essential elements:"
 - "A **name**, a meaningful reference to the pattern."
 - (名称：对模式的有意义的引用。)

- "A **description of the problem area** that explains **when** the pattern may be applied."
 - (问题描述：解释何时可以应用该模式的问题领域描述。)
- "A **solution description** of the parts of the design solution, their relationships and their responsibilities. It is a **template**... that can be instantiated in different ways."
 - (解决方案描述：设计解决方案的各个部分、关系和职责。它是一个可以在不同方式下实例化的模板。)
- "A **statement of the consequences**—the results and **trade-offs**—of applying the pattern."
 - (后果陈述：应用模式的结果和权衡。)

第一部分：第3章核心知识点梳理

1. 敏捷开发特征与原则 (Characteristics & Principles)

- **敏捷开发的特征 (Characteristics):**
 - **交织进行 (Inter-leaved):** 程序规格说明、设计和实现是交织在一起的，不是分阶段的。
 - **增量式 (Incremental):** 系统是一系列增量开发的。
 - **工具支持 (Tool support):** 广泛使用自动化测试工具等支持开发。
 - **文档最小化 (Minimal documentation):** 关注可工作的代码，而不是文档。
- **敏捷宣言/原则 (Agile Manifesto/Principles):**
 - **客户参与 (Customer involvement):** 客户应贯穿整个开发过程 (提供需求、评估增量)。
 - **拥抱变化 (Embrace change):** 预期需求会变化，设计系统以适应变化。
 - **增量交付 (Incremental delivery):** 软件分增量交付。
 - **保持简单 (Maintain simplicity):** 在软件和开发过程中都关注简单性，消除复杂性。
 - **人胜于过程 (People, not process):** 承认团队成员的技能，让他们自己决定工作方式。

2. 敏捷方法的适用性 (Applicability)

- **最适合:**
 - **中小型产品 (Small or medium-sized products)**。

- **定制系统 (Custom system development)**: 在组织内部开发, 客户承诺参与。
- **主要限制**: 对于大型系统 (Large-sized products) 或复杂的系统之系统, 传统的敏捷方法需要进行调整 (伸缩)。

3. 极限编程 (Extreme Programming, XP)

8. 核心实践 (Practices):

- **重构 (Refactoring)**: 所有开发者应在发现改进点时立即持续改进代码 (不需要等待急需时才做), 保持代码简单和可维护。这是让软件更易于修改的主要方法。
 - **结对编程 (Pair programming)**: 两人共用一台电脑。是一种非正式的评审机制 (Informal review), 鼓励重构, 支持代码共同拥有权。
 - **测试先行开发 (Test-first development)**: 在写代码前先写测试。
 - **增量规划 (Incremental planning) / 用户故事 (User stories)**:
- 需求以**用户故事 (User stories)** 或场景的形式表达, 写在卡片上。
 - 不使用详细的正式规格说明 (Detailed formal specification)。
 - 开发团队将故事分解为**任务 (Tasks)**。
 - **客户 (Customer)** 根据优先级和进度评估选择下一个版本包含的故事。
 - 客户在测试中的角色是帮助开发**验收测试 (Acceptance tests)**。

4. Scrum 方法 (Scrum Method)

Scrum 是一种敏捷项目管理框架。

- **术语 (Terminology)**:
 - **Sprint (冲刺)**: 一个开发迭代周期 (Development iteration), 通常 2-4 周。
 - **Product Backlog (产品待办事项)**: Scrum 团队必须解决的 "To do" 事项清单 (List of "to do" items)。
 - **Scrum (每日站立会议)**: 每日会议, 回顾进度, 优先级排序。
- **角色 (Roles)**:
 - **Product Owner (产品负责人)**: 确定产品功能或需求的**优先级 (Prioritize)**, 管理 Product Backlog。
 - **ScrumMaster (Scrum 主管)**: 负责安排会议 (Arranges daily meetings)、跟踪 Backlog、记录决

策、与外部干系人沟通 (Communicates with stakeholders)。**注意**: Scrum 中通常不设立传统的 "Project Manager", ScrumMaster 起协调作用, 但在一般敏捷项目管理语境下, 项目经理需确保按时按预算交付。

- **Development Team (开发团队)**: 自组织团队。

5. 敏捷方法的伸缩 (Scaling Agile Methods)

- **纵向伸缩 (Scaling up)**: 使用敏捷方法开发**大型系统 (Large systems)**, 这些系统太大, 无法由单个小分队开发。
- **横向伸缩 (Scaling out)**: 在大型公司中从专门团队扩展到更广泛地使用敏捷方法。
- **大型系统的特征**:
 - **Brownfield systems (棕地系统)**: 包含并与大量现有系统 (Existing systems) 交互的大型系统。
- **敏捷维护的问题 (Agile Maintenance Problems)**:
 - **缺乏产品文档 (Lack of product documentation)**。
 - **客户参与 (Customer involvement)**: 维护阶段客户很难持续参与。
 - **团队连续性 (Development team continuity)**: 团队解散后, 新成员难以理解系统。
- **技能要求**: 敏捷方法似乎在团队成员具有**相对较高的技能水平 (Relatively high skill level)** 时效果最好, 而不是技能参差不齐时。

1. 多选题

Which of the following options belong to the characteristics of agile development?

(以下哪些选项属于敏捷开发的特征?)

- A. Program specification, design and implementation are inter-leaved.
- B. The system is developed in a series of increments.
- C. Extensive tool support.
- D. Minimal documentation.
- **答案**: A, B, C, D
- **知识点**: Agile Development Characteristics (敏捷开发特征)
 - Specification, design and implementation are **inter-leaved** (交织进行)。
 - System is developed in **increments** (增量)。

-
- **Extensive tool support** (广泛的工具支持)。
- **Minimal documentation** (最小化文档)。

- **解析**: 这四个选项完整地概括了 PPT 中列出的敏捷开发特征。

2. 单选题

Which one of the options violates the principles of agile methods? (哪一个选项违反了敏捷方法的原则?)

- A. Avoid customer involvement throughout the development process.
- B. Expect the system requirements to change.
- C. The software is developed in increments.
- D. Focus on simplicity in both the software being developed and in the development process.
- **答案**: A
- **知识点**: Agile Principles (敏捷原则)
 - 原则之一是 **Customer involvement** (客户参与): Customers should be closely involved throughout the development process (客户应在整个开发过程中密切参与)。
- **解析**: 选项 A 说 "Avoid" (避免) 客户参与, 这与敏捷宣言中的 "客户协作" 直接背道而驰。

3. 单选题

Which option is false about the applicability of agile methods? (关于敏捷方法的适用性, 哪个选项是错误的?)

- A. small-sized products
- B. medium-sized products
- C. large-sized products
- D. custom systems
- **答案**: C
- **知识点**: Agile Method Applicability (敏捷方法适用性)
 - 敏捷方法最适合 **Small or medium-sized products** (中小规模产品) 和 **Custom system development** (定制系统开发)。
 - 对于 **Large-sized products** (大型产品), 传统的敏捷方法面临挑战, 需要进行 "Scaling up" (伸缩/扩展) 才能适用。
- **解析**: 标准敏捷方法 (如 Scrum/XP) 设计之初是针对小团队的, 不直接适用于大型系统, 因此 C 是错误的 (或

者说不完全适用的)。

4. 多选题

Which of the options belong to extreme programming practices?
(哪些选项属于极限编程的实践?)

- A. Refactoring
- B. Pair programming
- C. Incremental planning
- D. Test-first development
- **答案:** A, B, C, D (全选)
- **知识点:** Extreme Programming Practices (XP 实践)
 - Refactoring (重构)
 - Pair programming (结对编程)
 - Incremental planning (增量规划)
 - Test-first development (测试先行开发)
- **解析:** 这四项都是 XP 的核心实践 (Key practices)。

5. 单选题

For requirements engineering in extreme programming (XP), which one of the statements is false?
(对于极限编程 (XP) 中的需求工程, 哪项陈述是错误的?)

- A. A customer or user is part of the XP team and is responsible for making decisions on requirements.
- B. User requirements are expressed as detailed formal specification.
- C. The requirements are written on cards and the development team break them down into implementation tasks.
- D. The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.
- **答案:** B
- **知识点:** User Stories for Requirements (用户故事作为需求)
 - 在 XP 中, 用户需求表达为 **User stories** (用户故事) 或 scenarios, 而不是 **Detailed formal specification** (详细的正式规格说明)。
- **解析:** XP 强调文档最小化, 不写正式的规格说明书, 而是用故事卡片 (Story cards) 代替。

6. 判断题

In extreme programming, the programming team look for possible software improvements and make these improvements (refactoring) only when there is immediate need for them.

(在极限编程中, 编程团队寻找可能的软件改进, 并且仅在有必要需要时才进行这些改进 (重构)。)

- **答案:** False (✗)
- **知识点:** Refactoring (重构)
 - XP 主张: Make these improvements **even where there is no immediate need for them** (即使没有迫切需要, 也要进行改进)。
- **解析:** 重构是持续性的活动, 目的是保持代码整洁, 而不是等到不得不改的时候才改。

7. 单选题

In extreme programming, which is the main method by which the changes are easier to make to software?
(在极限编程中, 使软件更容易修改的主要方法是什么?)

- A. Using the most appropriate programming language.
- B. Keeping the customer involved.
- C. Refactoring
- D. Pair-programming
- **答案:** C
- **知识点:** Refactoring (重构)
 - Refactoring improves the software structure and readability and so **changes are easier to make** (重构改进了软件结构和可读性, 因此更容易进行修改)。
- **解析:** 通过不断重构, 防止代码腐烂, 从而降低后续修改的难度。

8. 判断题

In test-driven development, writing tests before code.
(在测试驱动开发中, 在写代码之前编写测试。)

- **答案:** True (✓)
- **知识点:** Test-first Development (测试先行开发)
 - Writing tests **before** code clarifies the requirements to be implemented (在代码前编写测试可以澄清要实现的需求)。
- **解析:** 这是 TDD (Test-Driven Development) 的核心定义: 红 (写失败测试) -> 绿 (写代码通过) -> 重构。

9. 填空题

The role of the customer in the testing process is to help develop [填空 1] tests for the stories that are to be implemented.
(客户在测试过程中的角色是帮助为要实现的故事开发____测试。)

- **答案:** acceptance / 验收

- **知识点:** Customer Involvement in Testing (客户参与测试)
 - The role of the customer... is to help develop **acceptance tests** (验收测试) for the stories。
- **解析:** 验收测试是用来确认系统是否满足客户需求的, 因此需要客户参与开发。

10. 单选题

Which statement is false about pair-programming?
(关于结对编程, 哪项陈述是错误的?)

- A. The same pair do not always program together, and pairs are created dynamically.
- B. Formal studies of the value of pair programming have had mixed results.
- C. It acts as an informal review process because each line of code is looked at by at least two people.
- D. It discourages refactoring which is used to improve the software structure.
- **答案:** D
- **知识点:** Pair Programming (结对编程)
 - 优点包括: Supports collective ownership (共同拥有权), Acts as an informal review (非正式评审), **Encourages refactoring** (鼓励重构)。
- **解析:** 结对编程实际上是**鼓励**重构的, 因为有同伴的支持, 开发者更有信心去改进代码结构。D 选项说 "Discourages" (阻碍/不鼓励) 是错的。

11. 单选题

In an agile project, it is whose responsibility to manage the project so that the software is delivered on time and within the planned budget?
(在敏捷项目中, 管理项目以确保软件按时并在计划预算内交付是谁的责任?)

- A. The development team members
- B. The project manager
- C. The customer
- D. The users
- **答案:** B
- **知识点:** Agile Project Management (敏捷项目管理)
 - PPT 原文: The principal responsibility of software **project managers** is to manage the project so that the software is delivered on time and within the planned budget.

- **解析**：虽然敏捷强调自组织，但从项目管理的宏观角度（特别是 PPT 第 3 章 Agile Project Management 部分的定义），按时按预算交付依然是“项目经理”这一角色的主要职责。在 Scrum 中这部分职责可能分散或由 ScrumMaster 协助，但就一般定义而言，选 B。

12. 单选题

The list of “to do” items which the Scrum team must tackle is called (Scrum 团队必须处理的“待办事项”清单称为)

- A. teamwork
- B. backlog
- C. sprint
- D. scrum
- **答案**： B
- **知识点**： Scrum Terminology (Scrum 术语)
 - **Product backlog**: This is a list of “to do” items which the Scrum team must tackle (这是 Scrum 团队必须处理的待办事项清单)。
- **解析**：Backlog (积压工作/待办事项) 是 Scrum 的核心工件之一。

13. 填空题

In scrum agile method, a development iteration is called a [填空 1]. (在 Scrum 敏捷方法中，一个开发迭代称为____。)

- **答案**： Sprint / sprint / 冲刺
- **知识点**： Sprint (冲刺)
 - **Definition**: A development iteration. Sprints are usually 2-4 weeks long (定义：一个开发迭代。通常 2-4 周长)。
- **解析**：Sprint 是 Scrum 中迭代的专有名称。

14. 单选题

In a scrum sprint cycle, who prioritizes the items on the product backlog to define the most important items to be developed? (在 Scrum 冲刺周期中，谁对产品待办事项列表中的项目进行优先级排序，以定义要开发的最重要项目？)

- A. The product owner
- B. The project manager
- C. The project developers
- D. The scrum master
- **答案**： A
- **知识点**： Product Owner (产品负责人)

- 职责： Identifies product features or requirements, **prioritize** these for development (识别产品特性或需求，并为开发进行优先级排序)。
- **解析**：Product Owner 代表客户利益，负责决定做什么以及做的顺序。

15. 单选题

In a scrum project, who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with stakeholders outside of the team?

(在 Scrum 项目中，谁负责安排每日会议、跟踪待办事项、记录决策、根据待办事项衡量进度并与团队外部的干系人沟通？)

- A. The team secretary
- B. The scrum master
- C. The product owner
- D. The stakeholder
- **答案**： B
- **知识点**： ScrumMaster (Scrum 主管)
 - 职责： The “Scrum master” is a facilitator who **arranges daily meetings**, tracks the backlog... and **communicates** with stakeholders outside of the team.
- **解析**：ScrumMaster 是团队的保护者和协调者，负责流程的顺利进行和外部沟通。

16. 判断题

“Scaling out” agile methods means to handle the development of large systems that are too big to be developed by a single small team.

(“Scaling out” 敏捷方法意味着处理那些太大而无法由单个小分队开发的大型系统的开发。)

- **答案**： False (✕)
- **知识点**： Scaling Up vs Scaling Out (纵向伸缩 vs 横向伸缩)
 - **Scaling up (纵向伸缩)**: Concerned with using agile methods for developing large software systems (用于开发大型系统)。
 - **Scaling out (横向伸缩)**: Concerned with how agile methods can be introduced across a large organization (如何在整个大型组织中推广敏捷方法)。
- **解析**：题目描述的是 **Scaling up** 的定义，而不是 Scaling out。

17. 多选题

Which are the three key problems in agile maintenance? (敏捷维护的三个关键问题是什么？)

- A. Lack of product documentation.
- B. Lack of contracts.
- C. Customer involvement.
- D. Development team continuity.
- **答案**： A, C, D
- **知识点**： Agile Maintenance Problems (敏捷维护问题)
 - - **Lack of product documentation** (缺乏产品文档)。
 - - **Keeping customers involved** (保持客户参与)。
 - - **Development team continuity** (开发团队连续性)。
- **解析**：这也是必考点。敏捷注重代码轻文档，导致维护难；客户在维护期难参与；人员流动导致知识流失。

18. 判断题

It is sometimes argued that agile methods require higher skill levels than plan-based approaches.

(有时人们认为，敏捷方法比基于计划的方法需要更高的技能水平。)

- **答案**： True (✓)
- **知识点**： Agile vs Plan-based (敏捷 vs 计划驱动)
 - 原文： It is sometimes argued that agile methods require **higher skill levels** than plan-based approaches (有时认为敏捷方法比基于计划的方法需要更高的技能水平)。
- **解析**：因为计划驱动方法中，程序员只需把详细设计翻译成代码；而敏捷开发中，开发者需要自己做设计、测试和实现，对个人能力要求更高。

19. 单选题

Large systems that include and interact with a number of existing systems, are called:

(包含并与许多现有系统交互的大型系统称为：)

- A. integrated systems
- B. systems of systems

- C. greenfield systems
- D. brownfield systems
- 答案: D
- 知识点: **Brownfield Systems (棕地系统)**
 - Large systems are **brownfield systems**; that is, they include and interact with a number of existing systems (大型系统通常是棕地系统, 即包含并与现有系统交互)。
- 解析: Greenfield (绿地) 指从零开始的全新开发; Brownfield (棕地) 指在现有基础上开发, 受限于遗留系统。

20. 判断题

Agile methods seem to work best when team members have a wide range of skills and abilities.

(当团队成员具有广泛的技能和能力 (水平参差不齐) 时, 敏捷方法似乎效果最好。)

- 答案: False (✖)
- 知识点: **Agile Team Skills (敏捷团队技能)**
 - Agile methods seem to work best when team members have a **relatively high skill level** (当团队成员具有相对较高的技能水平时, 敏捷方法效果最好)。
- 解析: 如果团队技能水平差异过大 (wide range), 特别是包含大量新手, 敏捷方法的